

UNIVERSITY OF TRENTO

University of Trento
Department of Industrial Engineering

Master's Degree in
Mechatronics Engineering

Master's Thesis

Safe LiDAR-based Navigation of Unknown Environments with Model Predictive Control

Supervisor
Prof. Andrea Del Prete

Co-supervisor
Prof. Elias Fontanari

Student
Claudia Gava

Academic Year 2025/2026

Acknowledgements

First and foremost, I would like to thank Professor Andrea Del Prete, my thesis supervisor, for his availability and the careful attention with which he guided every stage of this work. His suggestions brought greater rigor to the writing, and his expertise was fundamental to the evolution of the entire research.

I also wish to thank my co-supervisor, Elias Fontanari, who, in addition to following the entire development of this thesis, actively participated in training the DeepReach network for the reliability comparison.

A special thanks goes to my family, who gave me the opportunity to undertake this journey, always supporting me and showing pride in the results I have achieved. I tried to be as independent as possible during these years, but always with the comforting awareness that, had I needed it, you would have been there to help me.

A heartfelt thank you to Marco, who during these years had to put up with me countless times when I was stressed over studies, projects, or exams, and who proofread the entire thesis not once, but twice. You have been fundamental.

I would like to thank all the friends who shared these years with me and with whom I lived wonderful experiences. A special thanks to Andrea and Gabriele for their support, both technical and psychological, in solving the countless problems that arose along the way.

Finally, my gratitude goes to all those who, in different ways, contributed to this milestone. I hope you are as proud of me as I am of myself.

Abstract

In the field of autonomous robotics, safe navigation in unknown environments represents a fundamental challenge, especially for dynamically complex and underactuated systems such as quadrotor UAVs. Traditional navigation methods based on Model Predictive Control (MPC) offer excellent performance in trajectory tracking but suffer from inherent limitations related to recursive feasibility, the myopia of the prediction horizon, and high computational costs.

This thesis extends the Viability Boundary Optimal Control (VBOC) method to UAVs, defining Recoverability Boundary Optimal Control (RBOC) and introducing the concept of a "recovery box": a parameterized rectangular volume that models the physical space required for the vehicle to stop safely. By solving a series of offline Optimal Control Problems (OCPs), a dataset was generated to train the Neural Network. The network learns to predict the dimensions of the smallest possible recovery box based exclusively on the drone's kinematics (velocities and attitudes). The work proposes a novel framework that integrates optimal control (MPC) with Neural Network learning, to guarantee safe navigation relying on local perception via a LiDAR sensor (MPC-RBOC).

During flight, the real-time perception of the unknown environment is achieved through LiDAR mapping, which extracts a collision-free "safe-box" using an iterative Warm-Start approach for generic polygons. The Neural Network is then integrated into the MPC as a terminal constraint: safety is mathematically guaranteed by enforcing that the stopping space predicted by the network is always strictly contained within the free space detected by the sensor.

The system was initially implemented for a two-dimensional model (bicopter) and subsequently validated in its three-dimensional extension (quadcopter). Tests demonstrate that the proposed system (RBOC-MPC) significantly outperforms existing methods. In the safety validation, the Neural Network recorded a drastically lower number of missed alarms (unpredicted collisions) compared to complex state-of-the-art methods based on Hamilton-Jacobi reachability (DeepReach). Finally, in complex local path planning scenarios, the RBOC-MPC resolved the deadlock and infeasibility issues typical of Standard MPC (Std-MPC), while proving to be substantially more efficient and effective than classical reactive algorithms such as Artificial Potential Fields (APF).

Contents

1 Introduction **1**

2 Theory **3**
2.1 MPC . . . 3
2.1.1 Challenges for MPC . . . 3
2.2 Neural Network . . . 5
2.2.1 Training . . . 5
2.3 APF . . . 6

3 System Modeling **7**
3.1 Quadrotor Dynamics . . . 7
3.2 Bicopter Dynamics . . . 9
3.3 Fundamental parameters of the dynamic model . . . 9

4 Method Definition **11**
4.1 Viability Kernel . . . 11
4.1.1 Hamilton-Jacobi Reachability . . . 11
4.1.2 Viability Boundary Optimal Control (VBOC) . . . 11
4.2 Neural Network training . . . 14
4.3 Safe Navigation with MPC . . . 15
4.3.1 RBOC-MPC . . . 15
4.3.2 Constraints formulation for Local Path Planning . . . 16
4.4 Enviroment Mapping . . . 18
4.4.1 Slab Method . . . 18
4.4.2 Safe-box extraction . . . 19
4.4.3 Generalization of polygonal obstacle mapping in 2D . . . 21

5 Implementation, results and tests **23**
5.1 Safe Set Generation via RBOC . . . 23
5.2 Neural Network training and validation . . . 26
5.2.1 Safety Validation: Comparison with DeepReach . . . 26
5.3 Preliminary Evaluation in Global Path Planning (RBOC-MPC vs Std-MPC) . . . 29
5.3.1 Test Parameters Definition . . . 29
5.3.2 Single flight test . . . 30
5.3.3 Multiple flight tests . . . 32
5.4 Evaluation in Local Path Planning (LiDAR mapping tests) . . . 35
5.4.1 Preliminary Tests on axis-aligned obstacles . . . 35

5.4.2 Tests in more complex environments and problem analysis 35
5.4.3 Analysis on Multiple Cases for generalized mapping 40
5.5 Validation of definitive mapping method 47
5.5.1 Instability of Case 6 and improvement of the Warm Start method . 47
5.5.2 Comparison with other state-of-the-art methods 52
5.5.3 Dynamic Environments 56

6 3D Adaptment **57**
6.1 Redefinition of the learning dataset, NN training and validation 57
6.2 RBOC-MPC application in 3D environments 58
6.2.1 Preliminary tests on axis-aligned obstacles 58
6.2.2 Generalization of polygonal obstacle mapping in 3D 59

7 Conclusions **61**

Chapter 1

Introduction

In the field of robotics today, safety management is of paramount importance, given the increasing interaction between humans and robots and the goal of making the latter increasingly autonomous even in unknown environments. In the state of the art, there are various methods to handle the macro-problem of obstacle avoidance [1]: the simplest methods are based on immediate reactive responses, which are often difficult to tune in their defining parameters and somewhat inefficient in their results; more advanced methods rely on Model Predictive Control (MPC) or Machine Learning techniques, such as Neural Networks and Deep Reinforcement Learning. While safely avoiding obstacles is a limitation in itself, the very perception of the impediment can represent a challenge. In the field of path planning, a preliminary distinction is made between Global and Local path planning, assuming that the moving agent has either a global or partial (local) perception of the environment and surrounding obstacles, respectively. Global perception requires the robot to be provided with a map in advance to understand the environment and navigate, making the environment no longer unknown. However, this can be unfeasible in environments where signals such as GPS or satellite data cannot be detected. Therefore, it becomes necessary to rely on local path planning, equipping the robot with sensors capable of mapping the surrounding environment at run-time. When selecting the sensors to use, cameras are very often preferred for environment mapping through image comparison. This type of sensor is severely affected by environmental lighting conditions and the speed at which the vehicle travels, failing to guarantee high resolution in all cases. Active sensors like LiDAR, which calculate depth via laser pulses, are instead independent of lighting conditions and offer more robust and precise measurements for real-time mapping, thus proving more suitable for safety purposes.

The presented work focuses on the safety management of quadrotor drones, initially modeled and studied in the simplified 2D version of a bicopter. To ensure flight safety, the Viability Boundary Optimal Control (VBOC) method, previously developed for robotic manipulators [2], [3], [4], has been adapted. Since the drone is an underactuated system, this approach was extended in this work by introducing the concept of a "recovery box" (stop box): an obstacle-free rectangular volume whose scale is minimized around the drone's braking trajectory. By solving this optimization problem for various initial conditions, a sufficiently large dataset was generated to train a Neural Network. The network learns the minimum dimensions of this braking volume and imposes them as a terminal constraint within an MPC controller, ensuring that the drone can always safely stop in front of an obstacle. Concurrently, real-time environment perception is handled via a LiDAR sensor that extracts a local rectangular free space (defined as a "safe-box"). Flight safety is ultimately mathematically guaranteed by enforcing that the braking box

predicted by the Neural Network is always strictly contained within the safe-box detected by the LiDAR.

Thesis Structure

The thesis is organized as follows: Chapter 2 introduces the theoretical concepts of Model Predictive Control (MPC), Neural Networks, and the Artificial Potential Field (APF) algorithm; Chapter 3 describes the dynamic model of the quadrotor and its simplified bicopter version; Chapter 4 formally defines the proposed methodology, starting from the adaptation of the Viability Kernel and the training of the Neural Network, up to the extraction of the safe-box via LiDAR and its application in the MPC; Chapter 5 presents the practical implementation and a detailed analysis of the results, validating the proposed method through multiple navigation tests and comparing it with other approaches; Chapter 6 expands the mapping algorithm and system validation to three-dimensional space; finally, Chapter 7 summarizes the conclusions of the work.

Chapter 2

Theory

2.1 MPC

Model Predictive Control (MPC) consists of a finite-horizon discrete N-steps Optimal Control Problem (OCP), where the current state x_i is used as the initial state x_0 to solve the problem (Eq. 5):

$$\begin{aligned} x^*, u^* = \underset{x,u}{\operatorname{argmin}} \quad & \sum_{i=0}^{N-1} l(x_i, u_i) \\ \text{subject to} \quad & x_{i+1} = f(x_i, u_i) \quad \forall i = 0 \dots N-1 \\ & x_{i+1} \in \mathcal{X}, u_i \in \mathcal{U} \\ & x_0 = x_{curr} \end{aligned} \tag{Eq. 1}$$

At each state variation, the optimal control problem is reiterated and the optimal trajectory is recalculated, starting from the current state. Trajectories can vary significantly between one iteration and the next (predicted vs actual trajectory) due to the shrinking horizon: at the i-th step, the solver actually sees the tuple (t_i, x_i, N_i) and at the next iteration $(t_{i+1}, x_{i+1}, N_{i+1})$, where $N_{i+1} = N_i - \Delta step < N_i$. This is why each single optimization only saves the trajectories up to the next step, disregarding what is predicted for the future.

2.1.1 Challenges for MPC

In the definition of an MPC, certain characteristics are particularly important and represent real challenges for the problem: feasibility, stability, and computational time.

INFINITE HORIZON If $N = \infty$, the optimization would automatically no longer see a significant decrease in the horizon between one step and the next, meaning $N_i = N_{i+1}$. This ensures that the predicted trajectory perfectly coincides with the actual one, solving both stability and feasibility problems.

However, having an infinite horizon is not an easily viable solution. For this reason, a terminal cost and a terminal constraint at a sufficiently large finite horizon are often added to the problem to mimic an infinite one. Terminal constraints $x_N \in \mathcal{X}_F$ are used to manage feasibility, while the terminal cost $l_f(x_N)$ is used for stability.

FEASIBILITY For the management of the feasibility problem, as previously mentioned, it is convenient to work on the definition of the terminal constraints $\mathcal{X}_f$.

Theorem 1. *If $\mathcal{X}_f$ is a control invariant set, MPC has recursive feasibility.*

Definition 1. *A set $\mathcal{S}$ is control invariant if $\forall x \in \mathcal{S}\ \exists u \in \mathcal{U}$ s.t. $f(x,u) \in \mathcal{S}$.*

An example of a control invariant set for manipulators is the zero-velocity set $\mathcal{S} = \{(q,0) \mid q_{min} \leq q \leq q_{max}, \forall q \in \mathbb{R}^n\}$. Given the system dynamics and a finite horizon, imposing a zero terminal velocity indirectly bounds the initial velocities of the system.

To avoid excessively limiting performance, one solution is to use a broader version of the control invariant set $\mathcal{S}$, defined as the N-step backward reachable set $\beta_N(\mathcal{S})$.

Definition 2. *$\beta_N(\mathcal{S})$ is composed of all states $\bar{x}$ from which $\mathcal{S}$ can be reached within a limited time interval Δt.*

To check if $\bar{x} \in \beta_N(\mathcal{S})$, the following problem (Eq. 2) must be solved:

$$\begin{aligned} \underset{x,u}{\text{minimize}} \quad & 1 \\ \text{subject to} \quad & x_{i+1} = f(x_i, u_i) \quad \forall i = 0 \dots N \\ & x_{i+1} \in \mathcal{X}, u_i \in \mathcal{U} \\ & x_0 = \bar{x} \\ & x_N = x_{N+1} \end{aligned} \tag{Eq. 2}$$

STABILITY The stability of a problem can be managed by reducing the cost function to a Lyapunov Function.

Definition 3. *$V(x) \in \mathbb{R}^n$ is an Exponential Lyapunov Function if $\exists\ \alpha_1, \alpha_2, \alpha_3 > 0$ st $\forall x \in \mathcal{X}:\ \alpha_1|x| \leq V(x) \leq \alpha_2|x|$ and $V(x_1) - V(x_0) \leq -\alpha_3|x|$.*

Theorem 2. *If there exists a Lyapunov Function, the origin is exponentially stable in $\mathcal{X}$.*

Consider the Value Function $V^*(x)$, which is the optimal cost function, evaluated between two successive time instants: $V^*(x_0)$ and $V^*(x_1)$. However, knowing that the MPC recalculates the trajectory at each step, directly comparing $V^*(x_0)$ and $V^*(x_1)$ would be relatively meaningless. For this reason, we consider the exact same trajectory calculated at $t = 0$, simply shifted in time as $V(x_1)$, knowing with certainty that $V^*(x_1) \leq V(x_1)$:

$$\begin{aligned} V^*(x_0) &= \sum_{i=0}^{N-1} l(x_i, u_i) + l_F(x_N) \\ V(x_1) &= \sum_{i=1}^{N} l(x_i, u_i) + l_F(x_{N+1}) \end{aligned} \tag{Eq. 3}$$

from which, by eliminating common terms:

$$V(x_1) - V^(x_0) = l(x_N, u_N) + l_F(x_{N+1}) - l_F(x_N) - l(x_0, u_0) \tag{Eq. 4}$$

To verify that the value function $V^(x)$ is a Lyapunov Function, the following assumptions must hold simultaneously:

$$\begin{aligned} & f(0,0) = 0,\ l(0,0) = 0,\ l_F(0,0) = 0 \\ & \forall x \in \mathcal{X}_F, \exists u \in \mathcal{U} : l_F(f(x,u)) - l_F(x) + l(x,u) \leq 0 \\ & \exists \alpha_1 > 0 : \forall x \in \mathcal{X}_F, \forall u \in \mathcal{U} \text{ it has } l(x,u) \geq \alpha_1|x| \end{aligned} \tag{Eq. 5}$$

COMPUTATIONAL TIME In the context of Model Predictive Control (MPC), computational time is a critical aspect, since an Optimal Control Problem (OCP) must be solved at every sampling instant. In general, computational times are proportional to the horizon N, and therefore a trade-off is required to achieve good results in terms of feasibility, stability, and computational time.

Several practical strategies exist to reduce computation time and make the algorithm more efficient. A first technique consists of using the results of the previous OCP to initialize the current problem (Warm Start). Similarly, it can be useful to perform a shift of the optimal control sequence calculated at time $k-1$, using it as an initial guess at time k: $u_k = u_{k-1}$. A further simplification consists of using a zero initialization for some variables at the last time step: $u_{n-1} = 0$. If a certain degree of uncertainty is allowed in the resolution, the number of iterations the solver performs to reach convergence can be reduced, accepting an approximate result. At the hardware level, classical techniques involve using computational accelerators (e.g., GPUs) and parallelizing the compiler's computations.

2.2 Neural Network

Neural Networks are powerful parametric functions whose architecture is inspired by the functioning of the human brain.

The network is composed of an input layer and an output layer, while in the intermediate section, there can be one or more hidden layers. Each layer consists of neurons, and the network is called fully connected because every neuron in a layer is connected to all the neurons in the subsequent layer. Each neuron receives various inputs x_i to produce a single output y and can be described by the following nonlinear function, defined as the activation function:

$$y = g(\sum_i w_i x_i + w_0) \qquad \text{(Eq. 6)}$$

where w_i are the weights assigned to each input x_i, and w_0 is an initial bias. As the activation function $g(x)$, the hyperbolic tangent (tanh), ReLU, or GeLU are typically used.

2.2.1 Training

To be trained, Neural Networks must undergo a training process that consists of learning the weights w_i that define them.

The classical method for managing the training is through a supervised learning process, providing the network with a generic state x and the corresponding expected output y_{target}. All of this translates into solving an unconstrained optimization problem (Eq. 7):

$$\underset{w}{\text{minimize}} \sum_i ||\phi(x_i, w) - y_{target}||^2 \qquad \text{(Eq. 7)}$$

where the cost function is defined as the loss function $J(w) = \sum_i ||\phi(x_i, w) - y_{target}||^2$ and is simply a mean square error between the Neural Network prediction $\phi(x, w)$ and the set target y_{target}.
The minimization is typically solved using the numerical gradient descent method:

$$w_{next} = w - \alpha \nabla J \qquad \text{(Eq. 8)}$$

where α is the learning rate.

However, there are also other more complex methods that approximate Newton's numerical methods (e.g., Adam).
In general, the greater the number of data points provided to the network, the better the learning will be. However, this makes the calculation of the gradient ∇J computationally expensive. This is why standard practice involves using only a subset of the entire dataset, defined as a minibatch. Furthermore, to avoid having a strong correlation in the buffers, the data is randomly sampled and "shuffled" together.

2.3 APF

The Artificial Potential Field (APF [5]) is an algorithm first implemented in 1986 by Oussama Khatib to handle path planning and collision avoidance for manipulators and mobile robots [6].

The basic principle of this method is that the subject moves within a force field, the resultant of which determines the direction of motion toward a target position x_{target}. To simplify the analysis, the moving subject and the target are described as point masses, while the obstacles are modeled as 2D circles.

The attractive potential field function and repulsive potential field function can be represented as:

$$\begin{aligned} U_{tot}(x) &= U_{att}(x) + U_{rep}(x) \\ U_{att}(x) &= \frac{1}{2} k_{att} ||x - x_{target}||^2 \\ U_{rep}(x) &= \begin{cases} \frac{1}{2} k_{rep} (\frac{1}{\rho} - \frac{1}{\rho_0})^2 & \rho \leq \rho_0 \\ 0 & \rho > \rho_0 \end{cases} \end{aligned} \tag{Eq. 9}$$

where k_{att} and k_{rep} are the attractive and repulsive gain coefficients respectively, ρ_0 represents the limit distance of the potential field influence, and ρ is the shortest distance to the obstacle, i.e., between the center of the circle and the position x. This specific formulation of the repulsive potential (known as the FIRAS function) ensures that both the potential function and its derivative are perfectly continuous at the boundary $\rho = \rho_0$, thus avoiding sudden step changes in force intensity that would make the control unstable.

The corresponding attractive and repulsive forces can be calculated from the gradients as follows:

$$\begin{aligned} F_{att}(x) &= -\nabla U_{att}(x) = -k_{att}(x - x_{target}) \\ F_{rep}(x) &= -\nabla U_{rep}(x) = \begin{cases} k_{rep} (\frac{1}{\rho} - \frac{1}{\rho_0}) \frac{1}{\rho^2} \frac{\partial \rho}{\partial x} & \rho \leq \rho_0 \\ 0 & \rho > \rho_0 \end{cases} \end{aligned} \tag{Eq. 10}$$

Considering the presence of multiple obstacles, the resulting total force is given by the vector sum of the individual components:

$$F_{tot}(x) = F_{att}(x) + \sum_{obs=1}^{n} F_{rep,obs}(x) \tag{Eq. 11}$$

The fundamental limitation of this method is related to the local minima problem. When the attractive and repulsive forces are perfectly equal in magnitude but opposite in direction (e.g., when the subject is perfectly aligned frontally with an obstacle), the resultant force becomes zero; in such a scenario, the system no longer advances and remains erroneously trapped in a stall point.

Chapter 3

System Modeling

3.1 Quadrotor Dynamics

The dynamics to be applied to VBOC have been redefined for the drone, considering it a rigid body with 6 DoF $q = [\zeta, \eta] = [x, y, z, \phi, \theta, \psi]$ [7]. All variables are implicitly functions of time t. The state defining the drone is $X = [q, \dot{q}] \in \mathbb{R}^{12}$.

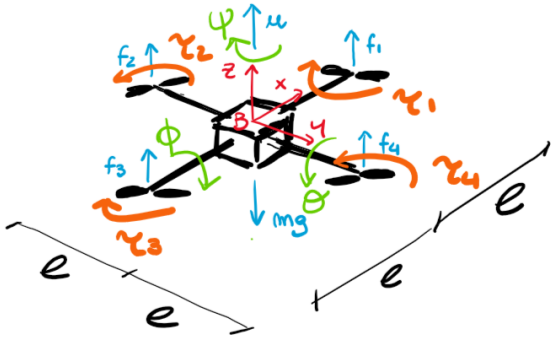


Figure 3.1: Quadrotor dynamics diagram

The drone's local reference frame (B) is mapped to the world reference frame (W) by pre-multiplying by the rotation matrix R_W:

$$R_W = R_z(\psi)R_y(\theta)R_x(\phi) = \begin{bmatrix} c\theta c\psi & c\psi s\theta s\phi - c\phi s\psi & s\phi s\psi + c\phi c\psi s\theta \\ c\theta s\psi & c\phi c\psi + s\theta s\phi s\psi & c\phi s\theta s\psi - c\psi s\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{bmatrix} \quad \text{(Eq. 1)}$$

where c and s are the abbreviated notations for cos and sin.

Rotational velocities are derived in the wrf knowing that

$$W_W = \frac{dR_W}{dt} R_W^{-1} \quad \text{(Eq. 2)}$$

and are then mapped back to the body reference frame

$$W_B = R_B W_W R_W = \begin{bmatrix} 0 & s\phi\theta' - c\theta c\phi\psi' & c\phi\theta' + c\theta s\phi\psi' \\ -s\phi\theta' + c\theta c\phi\psi' & 0 & -\phi' + s\theta\psi' \\ -c\phi\theta' - c\theta s\phi\psi' & \phi' - s\theta\psi' & 0 \end{bmatrix} \quad \text{(Eq. 3)}$$

where $R_B = [R_W]^T$.

Knowing that W is a skew matrix, the rotational velocities can be derived

$$W = \begin{bmatrix} 0 & -w_z & w_y \\ w_z & 0 & -w_x \\ -w_y & w_x & 0 \end{bmatrix} \quad \begin{aligned} w_x &= \phi' - s\theta\psi' \\ w_y &= c\phi\theta' - c\theta s\phi\psi' \\ w_x &= -s\phi\theta' - c\theta c\phi\psi' \end{aligned} \tag{Eq. 4}$$

from which $\Omega = W_\eta \eta'$:

$$\begin{bmatrix} w_x \\ w_y \\ w_z \end{bmatrix} = \begin{bmatrix} -s\theta & 0 & 1 \\ c\theta s\phi & c\phi & 0 \\ c\theta c\phi & -s\phi & 0 \end{bmatrix} \cdot \begin{bmatrix} \psi' \\ \theta' \\ \phi' \end{bmatrix} \tag{Eq. 5}$$

The equations of motion can be derived using the Euler-Lagrange method by calculating

$$\mathcal{L} = \mathcal{K} - \mathcal{P} \tag{Eq. 6}$$

where the potential energy is $P = mgz$ and the kinetic energy is given by

$$\mathcal{K} = \mathcal{K}_{trasl} + \mathcal{K}_{rot} = \frac{1}{2}\dot{\zeta}^T m\dot{\zeta} + \frac{1}{2}\Omega^T I\Omega = \frac{1}{2}m\dot{\zeta}^T\dot{\zeta} + \frac{1}{2}\dot{\eta}^T J\dot{\eta} \tag{Eq. 7}$$

where I is the diagonal inertia matrix with zero off-diagonal terms and $J = W_\eta^T I W_\eta$. The equation of dynamics is obtained from the Lagrangian as follows

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}}\right) - \frac{\partial \mathcal{L}}{\partial q} = \begin{bmatrix} F_\eta \\ \bar{\tau} \end{bmatrix} \tag{Eq. 8}$$

dividing the external actions into a translational and a rotational component:

$$F_\eta = R_W \hat{F} = \begin{bmatrix} T(c\phi c\psi s\theta + s\phi s\psi) \\ T(-c\psi s\phi + c\phi s\theta s\psi) \\ Tc\theta c\phi \end{bmatrix} \tag{Eq. 9}$$

where $\hat{F} = [0, 0, T]^T$, $T = \sum_{i=1}^{4} f_i = f_1 + f_2 + f_3 + f_4$ and $f_i = Kw_i^2$ for the single engine, while

$$\tilde{\tau} = \begin{bmatrix} (f2 - f4)l \\ (f3 - f1)l \\ R \end{bmatrix} = \begin{bmatrix} \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} \quad and \quad \bar{\tau} = W_\eta^T \tilde{\tau} = \begin{bmatrix} -\tau_\phi s\theta + c\theta(\tau_\psi c\phi + \tau_\theta s\phi \\ \tau_\theta c\phi - \tau_\psi s\psi \\ \tau_\phi \end{bmatrix} \tag{Eq. 10}$$

with $R = \sum_{i=1}^{4} \tau_i = \sum_{i=1}^{4} \tau_i = -\tau_1 + \tau_2 - \tau_3 + \tau_4$ (two motors rotate in clockwise and the others two in counterclockwise).

The equations of motion assume Coriolis-centripetal forces to be zero and describe an underactuated system

$$M(q)\ddot{q} + g(q) = S(q)^T u \tag{Eq. 11}$$

with

$$M = \begin{bmatrix} mI_{3x3} & 0_{3x3} \\ 0_{3x3} & J \end{bmatrix} \quad g = \begin{bmatrix} 0 & 0 & mg & 0 & 0 & 0 \end{bmatrix}^T \quad u = \begin{bmatrix} T & \tau_\phi & \tau_\theta & \tau_\psi \end{bmatrix}^T$$

$$S^T = \begin{bmatrix} s\phi s\psi + c\phi c\psi s\theta & 0 & 0 & 0 \\ c\phi s\theta s\psi - c\psi s\phi & 0 & 0 & 0 \\ c\theta c\phi & 0 & 0 & 0 \\ 0 & -s\theta & c\theta s\phi & c\theta c\phi \\ 0 & 0 & c\phi & -s\phi \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} R_{13} & \\ R_{23} & 0_{3x3} \\ R_{33} & \\ 0_{3x1} & W_\eta^T \end{bmatrix} \tag{Eq. 12}$$

where q, $\dot{q}$ and u are limited:

$$\begin{aligned} q \in \mathcal{Q} &= [q \in \mathbb{R}^6 | q_{min} \leq q \leq q_{max}] \\ \dot{q} \in \dot{\mathcal{Q}} &= [\dot{q} \in \mathbb{R}^6 | \dot{q}_{min} \leq \dot{q} \leq \dot{q}_{max}] \\ u \in \mathcal{U} &= [u \in \mathbb{R}^4 | u_{min} \leq u \leq u_{max}] \end{aligned} \tag{Eq. 13}$$

3.2 Bicopter Dynamics

A first testing phase was carried out for the simplified 2D bicopter case; therefore, for the sake of completeness, a brief analysis of the dynamics in this case is also reported.

In a two-dimensional x-z space, the drone is assumed to be composed of a rigid body defined by a rod in which the motors are placed at a symmetric distance $\updownarrow$ from the center of mass. Both motors are assumed to have the same rotation. The system has 3 DoFs $q = [x, z, \theta]$, where the variables are implicit functions of time t. The thrust that accelerates the drone and the torque that allows it to rotate along the y-axes are respectively:

$$\begin{aligned} T &= f_1 + f_2 \\ \tau &= l(f1 - f2) \end{aligned} \tag{Eq. 14}$$

Continuing with the Lagrangian method, the potential energy of the system and the kinetic energy, which consists of a translational and a rotational component, can be calculated:

$$\begin{aligned} P &= mgz \\ K &= \frac{1}{2}m(\dot{x}^2 + \dot{z}^2) + \frac{1}{2}I\dot{\theta}^2 \end{aligned} \tag{Eq. 15}$$

By calculating the Lagrangian (Eq. 6) and its derivatives (Eq. 8), the equations of motion are derived:

$$\begin{cases} \ddot{x} = -\frac{U_1+U_2}{m}\sin(\theta) \\ \ddot{z} = \frac{U_1+U_2}{m}\cos(\theta) - g \\ \ddot{\theta} = \frac{l(U_1-U_2)}{I_{yy}} \end{cases} \tag{Eq. 16}$$

considering the projections of the thrust $T_x = -T\sin(\theta)$ and $T_z = T\cos(\theta)$.

3.3 Fundamental parameters of the dynamic model

The values of the fundamental parameters used in the models of the quadrotor drone and the bicopter—modeled as a 3D ellipsoid and a 2D ellipse, respectively—are summarized below in Tab. 3.1.

Description	**Parameter**	**Value**
Mass	$m_{bicopter}$ $m_{quadcopter}$	0.75 Kg 1.5 Kg
Inertia	I_{xx} I_{yy} I_{zz} $I_{xy} = I_{xz} = I_{yz}$	$7.75 \cdot 10^{-3}\ kg\ m^2$ $7.35 \cdot 10^{-3}\ kg\ m^2$ $12.55 \cdot 10^{-3}\ kg\ m^2$ $0\ kg\ m^2$
Length of motor arm	l	0.10 m
Thrust coefficient	cf	$1.5 \cdot 10^{-3}\ N/Hz^2$
Torque coefficient	ct	$4.59 \cdot 10^{-5}\ Nm/Hz^2$
Max rotation speed for motors	w_{max}	108 rad/s
Length of the ellipsoid's semi-axis	d_x d_y d_z	0.15 m 0.15 m 0.05 m

Table 3.1: Drone dimensional parameters

Chapter 4

Method Definition

4.1 Viability Kernel

To ensure safety in a dynamical system, most existing methods rely on the knowledge of safe set (a.k.a. control-invariant set). The largest control-invariant set is known as the viability kernel, and various methods exist for approximately computing it, depending on the complexity of the system being analyzed. For the case under analysis——the computation of a safe set for a drone——two different methods have been considered: Hamilton-Jacobi Reachability and the Viability Boundary Optimal Control method.

4.1.1 Hamilton-Jacobi Reachability

The first approach was based on Hamilton-Jacobi reachability, relying on specific studies conducted on robots of the same type [8, 9]. These methods are thought for computing reach-avoid sets. Given an avoid set $\mathcal{A}$ and a target set $\mathcal{T}$, the goal is to compute the set of states starting from which the system can reach $\mathcal{T}$ while avoiding $\mathcal{A}$. Such reach-avoid sets can be proven to be control invariant if $\mathcal{T}$ is control invariant. For the system under analysis, we can set $\mathcal{T}$ to be the set of collision-free equilibrium states, which is trivially control invariant.

However, this method proved to be largely unsuccessful for our specific use-case because it uses a zero-volume target set: equilibrium states must have exactly zero velocity, therefore the set has no interior. Previous works have instead focused on target sets with non-zero volume, considering regions of the state space that were solely bounded in terms of position. This is not compatible with our goal of guaranteeing safety: indeed, if the robot reaches a target set with a non-zero velocity, it will tend to move away from it. This could lead to impacts with adjacent obstacles, thereby compromising safety.

4.1.2 Viability Boundary Optimal Control (VBOC)

Because of the problems explained above, a second method had to be employed, reusing the VBOC algorithm developed for manipulators and adapting it for drones [10].

Original Algorithm

The algorithm developed for manipulators calculates the states $x^T = [p, v]$ located exactly on the boundary of the safe set ($\partial\mathcal{V}$), without iteratively defining the entire safe-volume. To define these boundary points, it solves an OCP where the initial state is only partially specified. Qualitatively, given a random initial position q_{init}, and a random direction

belonging to the robot's "velocity space," d, the algorithm calculates the maximum initial velocity norm v_{max} so that the robot can reach an equilibrium in $N-1$ time steps. By exploiting the properties of velocity scaling and star-convexity of the viability kernel of robot manipulators, every velocity obtained by scaling down v_{max} will automatically also be safe as initial state for the manipulator.

The OCP is defined as follows:

$$\begin{aligned} v_{max} = \max_{\{x_i\}_0^N, \{u_i\}_0^{N-1}} \quad & a^T x_0 \\ \text{subject to} \quad & x_{i+1} = f(x_i, u_i) \quad \forall i = 0, \dots, N-1 \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U} \quad \forall i = 0, \dots, N-1 \\ & S x_0 = s \\ & x_N = x_{N-1} \end{aligned} \tag{Eq. 1}$$

where the state is $x^T = [p, v]$, $a^T = [0, d]$, $S = \begin{bmatrix} I & 0 \\ 0 & I - dd^T \end{bmatrix}$ and $s^T = [q_{init}, 0]$.

Regarding the horizon N, the algorithm seeks to extend it as much as necessary to compute the infinite-step backward reachable set of the equilibrium states (which in the paper is supposed to be equal to the viability kernel).

The tuples formed by the initial position, the velocity direction and the maximum velocity norm calculated by the algorithm, define the boundary of the viability kernel: $[q, d, v_{max}]$. A Neural Network was then trained via supervised learning to predict v_{max} given the values $[q, d]$.

Variation on Lemma 3 of [10]

Star-convexity of the viability kernel with respect to velocities does not hold for underactuated systems, as the assumption that the robot is capable of compensating for gravity in every configuration q is no longer valid. Therefore, VBOC cannot be directly applied to underactuated robots, such as the quadcopter under analysis. In the following we extend VBOC making it applicable to underactuated systems that have to navigate unknown environments.

An obstacle-free volume around the robot is considered, modeled as a rectangular cuboid (i.e. a box), asymmetrical with respect to the drone's center of mass. This box, defined in the global reference frame, is parameterized by the vector $b \in \mathbb{R}^6$:

$$b = [b_x^+, b_y^+, b_z^+, b_x^-, b_y^-, b_z^-]^T \tag{Eq. 2}$$

Let us define the augmented state $x_{new} = [x, b]^T \in \mathbb{R}^{18}$, where the box parameters are concatenated with the dynamic state x. Since the box is defined in the global frame, these parameters have null dynamics ($\dot{b} = 0$). Instead of modeling collisions between the robot and the environment, we will now consider the simpler problem of maintaining the robot inside the obstacle-free box Box(b). In other words, we will consider the augmented state to be safe if and only if, starting from that state, the robot can reach an equilibrium state without leaving the obstacle-free box.

Lemma 1. *Given $x_{new} \in \mathcal{V}$, then for every scale factor $\alpha \geq 1$, the state obtained by expanding the box by alpha also belongs to $\mathcal{V}$: $x_\alpha = [x, \alpha b]^T \in \mathcal{V}$.*

Proof. By the definition of the Viability Kernel, if $x_{new} = [x, b]^T \in \mathcal{V}$, there exists an admissible control trajectory $u(t)$ for $t \in [0, t_f]$ such that $x_{new}(t_f) \in \mathcal{T}$ (where $\mathcal{T}$ is the

target set of safe equilibrium states, i.e., states with zero velocity and stable attitude), and the entire geometric footprint of the trajectory $p(t) \in \text{Box}(b)$ for $t \in [0, t_f]$. Multiplying the box dimensions by a scale factor $\alpha \geq 1$ defines a new spatial volume $\text{Box}(\alpha b)$. According to the elementary properties of Euclidean geometry, the expansion of a volume contains the original volume: $\text{Box}(b) \subseteq \text{Box}(\alpha b)$. Thus, $p(t) \in \text{Box}(\alpha b)$ for $t \in [0, t_f]$. Since the original trajectory belongs to the expanded box, the existence of a valid trajectory for the expanded state x_α is proven, which implies that $x_\alpha = [x, \alpha b]^T \in \mathcal{V}$. □

Modified Algorithm (RBOC)

The algorithm was adapted for drones, maintaining the basic structure of VBOC (Eq. 1), but transforming the objective into minimizing the dimensions of the box enclosing the drone, starting with a random pose and velocity:

$$\begin{aligned} \underset{\{x_i\}_0^N, \{u_i\}_0^{N-1}}{\text{minimize}} \quad & a^T x_0 \\ \text{subject to} \quad & x_{i+1} = f(x_i, u_i) && \forall i = 0, \ldots, N-1 \\ & x_{i+1} \in \mathcal{X}(b_{sampled}), \quad u_i \in \mathcal{U} && \forall i = 0, \ldots, N-1 \\ & S x_0 = s \\ & x_N = x_{N-1}, \end{aligned} \tag{Eq. 3}$$

where the state is augmented with the box scaling factor α, i.e. $x = [p, v, \alpha] \in \mathbb{R}^{13}$. The augment dynamics leave α constant throughout the horizon, $a^T = [0_{1\times 12}, 1]$ selects α from the augmented state, $S = [I_{12\times 12}, 0]$ and $s = [p_{sampled}, v_{sampled}]$ is the randomly sampled initial state. The state constraint set $\mathcal{X}(b_{sampled})$ constrains the state to lie inside a box parametrized by the vector $\alpha b_{sampled}$, where $b_{sampled}$ is a sampled vector with unitary norm: $\|b_{sampled}\| = 1$.

In the optimization, the dynamics were replaced with those of the drone (Eq. 12).

Among the constraints $x_i \in \mathcal{X}$, the constraint requiring drone's absolute position and footprint to remain strictly inside the box has been made explicit:

$$n_i^T p + \sqrt{n_i^T Q(x) n_i} \leq \alpha \cdot b_i \quad \forall i \in \{1, \ldots, 6\} \quad \text{in 3D} \tag{Eq. 4}$$

where $n_i^T p$ represents the projection of the drone's center of mass along the normal of the i-th wall of the box, $\sqrt{n_i^T Q(x) n_i}$ represents the distance from the center of mass to the edge of the drone's ellipsoid in the direction n_i, and $\alpha \cdot b_i$ is the absolute distance of the i-th box's face from the center of the reference frame.

Having defined the minimization only with respect to the scaling factor α, the solver will simply shrink the spatial volume of the box until the drone's braking trajectory strictly activates at least one of the constraints in(Eq. 4). Because the directional vector $b_{sampled}$ defining the shape of the box is considered fixed and normalized ($\|b_{sampled}\| = 1$), the optimized scaling factor α directly encapsulates the exact physical space (in meters) required for braking in the most critical direction (the active constraint).

In this new approach, the feasibility of the drone succeeding in the stopping maneuver is also influenced by the length of the horizon N. For this reason, the part of the solver dedicated to increasing the horizon was adapted so that N is not increased if the optimization succeeds, but rather if it fails: in other words, the solver attempts a resolution with an N_{start} (deliberately set low), and if it fails, it increases the horizon a finite number of times $N_{new} = N + \Delta N$; when and if the optimization converges, it saves, in addition to the state, the value of N for which it was feasible.

4.2 Neural Network training

Once the dataset defining the safe set $\mathcal{V}$ is obtained, a Neural Network is trained to learn the safe boundaries for the drone's braking maneuver. The network is coupled with the standard MPC as a terminal constraint, since the latter alone is inherently "short-sighted" when using short horizons N. The Neural Network serves exactly to widen the optimizer's view without having to increase N, which would otherwise raise the computational effort. By learning from the results of the offline optimizations via supervised learning, the network is able to provide an immediate estimation of the required braking space.

The Neural Network was defined using a multi-layer architecture configured with the parameters defined in Tab. 4.1:

Parameter	**Value**
learning rate	$1 \cdot 10^{-4}$
epochs	2000
batch size	1024
hidden size	1024
hidden layers	2
training ratio (%)	0.7
validation ratio (%)	0.2
test data (%)	0.1

Table 4.1: Parameters for Neural Network

The network takes as input a subset of the state, consisting of the angles and velocities, excluding both the position and the box dimensions. This choice is dictated by the fact that the minimum braking space depends exclusively on the drone's dynamics. Furthermore, given that the shape vector $b_{sampled}$ is defined as a fixed parameter with unitary norm, providing the surrounding room's geometry to the NN is unnecessary, as the optimized scaling factor α directly quantifies the physical braking distance.

Therefore, the scalar values of α obtained from the optimization (Eq. 3) are extracted and used directly as the target for the Neural Network, bypassing any additional geometric conversions.

4.3 Safe Navigation with MPC

The subsequent phase of the process involves using the trained Neural Network as support during an optimal control problem to simulate the drone's behavior in a real environment.

4.3.1 RBOC-MPC

The optimization is performed through an MPC (Eq. 5) where the state is defined solely by pose and velocities $x = [p, v] \in \mathbb{R}^{12}$ ($x = [p, v] \in \mathbb{R}^6$ in 2D), while the box parameters b (Eq. 2) and the scaling factor α_{real} are constant:

$$\begin{aligned} \underset{x,u}{\text{minimize}} \quad & \sum_{i=0}^{N-1} l(x_i, u_i) + l_N(x_N) \\ \text{subject to} \quad & x_{i+1} = f(x_i, u_i) && \forall i = 0, \dots, N-1 \\ & x_0 = x_{curr} \\ & x_{i+1} \in \mathcal{X}(b), \quad u_i \in \mathcal{U} && \forall i = 0, \dots, N-1 \\ & x_N \in \mathcal{X}_F \\ & \phi(\tilde{x}_N, b) \leq \alpha_{real}, \end{aligned} \tag{Eq. 5}$$

where the last two constraints ensure safety (i.e. recursive feasibility of the MPC problem). First, the terminal constraint set $\mathcal{X}_F$ restricts the state within the ranges in which the Neural Network was trained (velocities between ± 1 m/s and angles between $\pm\pi/2$ rad in 2D). Then ϕ is the trained Neural Network that predicts the minimum scaling factor of the box b necessary to stop the drone from the state x_N. This scaling must be smaller than α_{real}, which is the maximum scaling of the box b that prevents it from colliding with surrounding obstacles. Note that the Neural Network evaluates the braking distance based solely on the vehicle's dynamics; therefore, it does not take the entire terminal state x_N as input, but strictly the information regarding angles and velocities $\tilde{x}_N = (\theta_N, v_N)$.

The ratio between the different sides of the box is fixed and defined by the vector b. The computation of b and α_{real} must occur before solving the MPC problem. Their computation depends on the environment mapping technique, specifically whether the perception is global or local. When obstacles are known a priori in a global reference frame, the real available distance is simply the geometric difference between the obstacle's absolute coordinates and the drone's predicted position. Conversely, in the case of local real-time perception (e.g., using a LiDAR sensor), it is necessary to rigorously account for the spatial shift between the distance measured at the current instant and the drone's future predicted position. This specific complication will be thoroughly addressed in Sec. 4.3.2, detailing the constraint redefinition for the LiDAR mapping technique.

The running cost is defined as a weighted sum of the state error ϵ_x and the control error ϵ_u given a reference value, while the terminal cost considers only the position error with an additional gain $K = 20$:

$$\begin{aligned} l(x_i, u_i) &= \epsilon_{x_i}^T W_x \epsilon_{x_i} + \epsilon_{u_i}^T W_u \epsilon_{u_i} \\ l_F(x_N) &= K \cdot \epsilon_{x_N}^T W_x \epsilon_{x_N} \end{aligned} \tag{Eq. 6}$$

where $\epsilon_{x_i} = x_i - x_{ref} \in \mathbb{R}^{12}$ ($\epsilon_{x_i} \in \mathbb{R}^6$ in 2D), $\epsilon_{u_i} = u_i - u_{hover} \in \mathbb{R}^4$ ($\epsilon_{u_i} \in \mathbb{R}^2$ in 2D) with $u_{hover} = \frac{mg}{4c_f}$ ($u_{hover} = \frac{mg}{2c_f}$ in 2D).

The weights assigned to the state and the motors are W_x and W_u, respectively (in 2D Eq. 7). While the control weight W_u is deliberately kept low to give the solver maximum

freedom in using the motors, the weights on velocity and position in W_x are higher to center the target and penalize high speeds in its vicinity, preventing oscillatory behaviors and ensuring a precise and stable settling. Furthermore, the low weights on velocities make the drone more aggressive in its maneuvers, which highlights the "short-sightedness" of the Standard-MPC (Std-MPC) without terminal constraint.

$$W_x = \begin{bmatrix} 10^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 10^2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 20 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \qquad W_u = \begin{bmatrix} 10^{-4} & 0 \\ 0 & 10^{-4} \end{bmatrix} \quad \text{in 2D} \qquad \text{(Eq. 7)}$$

With the contribution of the Neural Network, the horizon N can be reduced, as well as the number of iterations of the solver. This allows the MPC to be solved in reasonable time, which is necessary considering that the drone should be able to perform the optimization while in flight.

4.3.2 Constraints formulation for Local Path Planning

When performing Local Path Planning, by no longer assuming a priori knowledge of the obstacle's position, the available free space must be calculated locally at each time step i. Consequently, the scaling factor defining the safe-box becomes a function of the current state.

The LiDAR sensor produces an asymmetric safe-box whose distances from the walls are measured in a local frame centered on the drone, with zero angle with respect to the World Frame. These distances can be parameterized by a vector b:

$$\begin{aligned} b &= [b_{x_min}, b_{x_max}, b_{z_min}, b_{z_max}] \quad \text{in 2D} \\ b &= [b_{x_min}, b_{x_max}, b_{y_min}, b_{y_max}, b_{z_min}, b_{z_max}] \quad \text{in 3D} \end{aligned} \qquad \text{(Eq. 8)}$$

Because the LiDAR evaluates the free space at the current instant, a direct comparison with the required braking space at the final predicted step x_N would be geometrically inconsistent. During the predictive horizon, the drone expects to move by an amount $dx = x_N - x_0$. If the constraints were evaluated using the original relative vector b, the bounding box would erroneously translate along with the drone.

To maintain geometric consistency, the distances from the walls should be dynamically updated at each step: $b_N = b - dx$.

However, to avoid introducing continuous recalculations within the optimizer, a mathematically equivalent but more efficient solution for the solver was chosen: switching to absolute coordinates (World Frame). Considering the drone's current position x_0, the boundaries of the relative box b are mapped to absolute coordinates, obtaining fixed spatial boundaries:

$$\begin{aligned} b_{abs} &= [x_{min}{}^{1}, x_{max}, z_{min}, z_{max}] \quad \text{in 2D} \\ b_{abs} &= [x_{min}, x_{max}, y_{min}, y_{max}, z_{min}, z_{max}] \quad \text{in 3D} \end{aligned} \qquad \text{(Eq. 9)}$$

Rather than being conservative by defining a single constraint through a global scale parameter, the constraint was decomposed, evaluating safety independently for each cardinal direction. For the maneuver to be safe, it is enforced that the actual distance between the predicted drone position and each absolute plane of the box is greater than or

[1] For example: $x_{min} = x_0 + b_{x_min}$, $x_{max} = x_0 + b_{x_max}$.

equal to the required braking space along the three axis in positive and negative direction $d_{pred} = \phi(\tilde{x}_N, b_{ref}) \cdot b_{ref}$:

$$\begin{cases} x_N - x_{min} \geq d_{pred,x^-} \\ x_{max} - x_N \geq d_{pred,x^+} \\ z_N - z_{min} \geq d_{pred,z^-} \\ z_{max} - z_N \geq d_{pred,z^+} \end{cases} \text{ in 2D} \qquad \begin{cases} x_N - x_{min} \geq d_{pred,x^-} \\ x_{max} - x_N \geq d_{pred,x^+} \\ y_N - y_{min} \geq d_{pred,y^-} \\ y_{max} - y_N \geq d_{pred,y^+} \\ z_N - z_{min} \geq d_{pred,z^-} \\ z_{max} - z_N \geq d_{pred,z^+} \end{cases} \text{ in 3D} \quad \text{(Eq. 10)}$$

4.4 Enviroment Mapping

To enable autonomous navigation in unknown environments, a real-time environment mapping strategy was implemented to simulate the operation of a LiDAR sensor. Specifically, this method was developed to map the environment at each iteration, which can theoretically be used to map both static and dynamic obstacles. For this first part, the analysis was limited to obstacles aligned with the World Reference Frame and the methods used started from this fundamental assumption both in the definition of the obstacles and in their mapping.

4.4.1 Slab Method

The perception of obstacles surrounding the drone is intended to simulate the operation of a LiDAR sensor (resolution $\Delta\theta = 1^\circ = 2\pi/360$ rad) and was implemented using an algorithm called Slab Method, which is widely used in computer graphics [11], [12].

For the sake of simplicity, it is assumed that the sensor coincides with the drone's center of mass. Each ray emitted by the sensor is modeled by the parametric equation of a half-line:

$$p(t) = p_0 + td \tag{Eq. 11}$$

where p_0 is the drone's position, d is the directional unit vector depending on the scanning angle, and $t \geq 0$ is the distance along the ray. A rectangular axis-aligned obstacle is encoded by a vector containing its limits:

$$\begin{aligned} obs &= [x_{min}, x_{max}, z_{min}, z_{max}] \quad \text{in 2D} \\ obs &= [x_{min}, x_{max}, y_{min}, y_{max}, z_{min}, z_{max}] \quad \text{in 3D} \end{aligned} \tag{Eq. 12}$$

To find where the ray hits the rectangle, the values of t for which the ray crosses the lines defining the obstacle boundaries (Eq. 12) are calculated by inverting Eq. 11.

Let t^x_{min} and t^x_{max} be the values of t for which the ray encounters the limit values of the obstacle in direction x, namely:

$$p^x(t^x_{min}) = x_{min}, \quad p^x(t^x_{max}) = x_{max} \tag{Eq. 13}$$

Similarly, we can generalize this definition to any direction $i \in \{x, y, z\}$, obtaining t^i_{min} and t^i_{max}. To check whether the ray effectively hits the obstacle we can verify if the entry and exit intervals along all directions overlap, i.e.:

$$\max_i(t^i_{min}) \leq \min_i(t^i_{max}) \tag{Eq. 14}$$

If this condition is met, we need to further check whether the impact distance, computed as $t_{enter} = \max_i(t^i_{min})$, is positive and less than the sensor's maximum range:

$$\text{distance} = t_{enter} = \max_i(t^i_{min}) \tag{Eq. 15}$$

The algorithm then saves the entire point cloud for each ray that hits an obstacle, extracting the coordinates $Q = p(t_{enter})$. To account for potential obstacles in-between rays, we model a circular obstacle of radius R, centered at each impact point Q, where R is computed so that the circular obstacles completely fill up the space between impact points:

$$R = t_{enter} \cdot \tan\left(\frac{\Delta\theta}{2}\right) \tag{Eq. 16}$$

4.4.2 Safe-box extraction

In the definitive formulation, the safe-box extraction relies on a Warm Start approach. The algorithm calculates the smallest possible safe-box containing the entire predicted trajectory extracted from the previous step $k-1$. Let the predicted trajectory at step $k-1$ be defined as the set of coordinates over the prediction horizon N:

$$\begin{aligned} traj_{pred} &= \left\{[x_{k-1}^0, z_{k-1}^0], \ldots, [x_{k-1}^N, z_{k-1}^N]\right\} \quad \text{in 2D} \\ traj_{pred} &= \left\{[x_{k-1}^0, y_{k-1}^0, z_{k-1}^0], \ldots, [x_{k-1}^N, y_{k-1}^N, z_{k-1}^N]\right\} \quad \text{in 3D} \end{aligned} \tag{Eq. 17}$$

The relative safe-box boundaries for the current step k, evaluated with respect to the drone's actual current position $p_k = [x_k, z_k]^T$ in 2D and $p_k = [x_k, y_k, z_k]^T$ in 3D, are calculated as:

$$box_k = \begin{bmatrix} \min_i(x_{k-1}^i) - x_k - M \\ \max_i(x_{k-1}^i) - x_k + M \\ \min_i(z_{k-1}^i) - z_k - M \\ \max_i(z_{k-1}^i) - z_k + M \end{bmatrix} \quad \text{in 2D} \qquad box_k = \begin{bmatrix} \min_i(x_{k-1}^i) - x_k - M \\ \max_i(x_{k-1}^i) - x_k + M \\ \min_i(y_{k-1}^i) - y_k - M \\ \max_i(y_{k-1}^i) - y_k + M \\ \min_i(z_{k-1}^i) - z_k - M \\ \max_i(z_{k-1}^i) - z_k + M \end{bmatrix} \quad \text{in 3D} \tag{Eq. 18}$$

where M is an arbitrary safety margin (e.g., 15 cm) added to prevent the created box from being too close to the predicted trajectory.

Since this initial safe-box has minimal dimensions, it must be iteratively expanded to grant the solver enough space for maneuvering. It is clarified that the safe-box calculation occurs prior to the MPC resolution so only the present states x_t and past states x_{t-1} are known. The directional vector is calculated as the difference between the predicted position and the current position of the system. In the absence of external disturbances, the state predicted at the previous step $x_{1|t-1}$ will perfectly match the current state x_t. For this reason, the prediction made for a horizon n at the past step, denoted as $x_{n|t-1}$, is considered:

$$v_{dir} = x_{pred} - x_{curr} \tag{Eq. 19}$$

The expansion order of the faces is managed by always moving the faces alternately along the axes and relying on two different criteria:

- Priority between parallel faces: based on the local target given by the directional vector v_{dir} (Eq. 19), an order is defined between the two faces on the same axis, assigning expansion priority to the face towards which the drone is heading.

- Priority between directional expansion axes: the choice to first execute the expansion along one axis rather than the others (e.g., x-axis before z-axis or vice versa in 2D) is dictated by the absolute distance to the final target.

Naturally, this directional expansion is physically bounded by the mapped environment. Once the spheres on the obstacle boundary are defined, the safe-box for the drone is defined using an iterative restriction algorithm that shrinks the expanding boundaries until there are no intersections with the boundary spheres [13, 14].

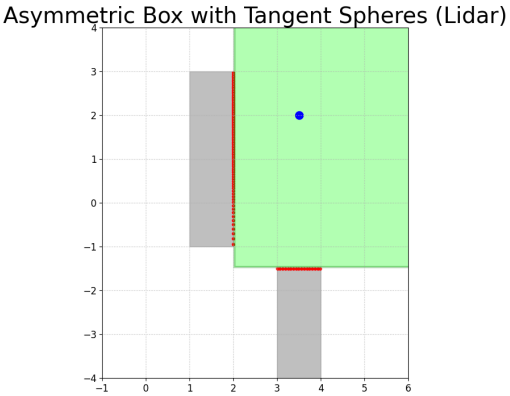


Figure 4.1: Obstacle mapping in 2D

The baseline implementation of this algorithm restricts the boundaries by evaluating possible contractions ("pushes") solely to maximize the geometric area (in 2D) or volume (in 3D) of the resulting box. However, a pure area or volume maximization heuristic sometimes results in sub-optimal or wrong spatial allocations for navigation, as the system tends to favor large expansions in directions opposite to the destination (Fig. 4.2).

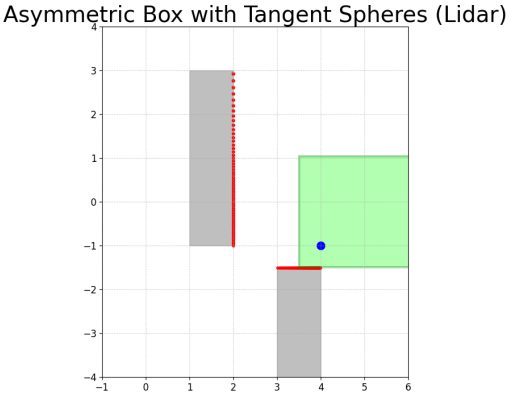


Figure 4.2: Incorrect calculation of the 2D safe box

To enforce a goal-oriented spatial allocation, the evaluation metric was extended to explicitly account for the target's location. The algorithm was modified to evaluate the expansion based on a weighted objective function, rewarding the expansion of the boundaries in the direction of the target:

$$\begin{aligned} Score &= Area + W \cdot (Bonus_X + Bonus_Z) \quad \text{in 2D} \\ Score &= Volume + W \cdot (Bonus_X + Bonus_Y + Bonus_Z) \quad \text{in 3D} \end{aligned} \tag{Eq. 20}$$

where W[2] is a user-defined weight.

[2]In 2D, $W = 15$ for axis-aligned obstacles, $W = 50$ for general obstacles.

4.4.3 Generalization of polygonal obstacle mapping in 2D

The Slab Method previously used with that specific resolution technique is only suitable for identifying obstacles aligned with the World Reference Frame. In order to map any polygon defined by straight line segments, it is necessary to extend the previous method with a more generic one [15].

Obstacles are defined by n vertices, which are connected to form closed polygons. It is assumed that the vertices of the polygons are defined sequentially:

$$b = [[x_1, z_1], \dots [x_{n-1}, z_{n-1}], [x_n, z_n]] \quad \text{in 2D} \tag{Eq. 21}$$

A segment is thus defined between two points A and B as $s = B - A = [s_x, s_z]^T$. Keeping the basic idea of finding the intersection between the LiDAR ray and the segments forming the edges of the obstacles (Eq. 11), two lines are drawn:

$$\begin{aligned} p(t) &= p_0 + td \quad \text{for the LiDAR} \\ S(u) &= A + us \quad \text{for the side of the box} \end{aligned} \tag{Eq. 22}$$

where $t \geq 0$ is the distance traveled along the LiDAR ray in the direction d, and $u \in [0, 1]$ indicates the displacement percentage along the segment s connecting points A and B.

The impact occurs when the two lines intersect, i.e., $p_0 + td = A + us$, for some values of t and u. The single equation can be rewritten as a system by considering the individual components along the axes of the reference frame:

$$\begin{cases} -td_x + us_x = p_{0,x} - A_x = \Delta x \\ -td_z + us_z = p_{0,z} - A_z = \Delta z \end{cases} \quad \text{in 2D} \tag{Eq. 23}$$

Once the solutions for the unknowns t and u are obtained from the system, the results are considered valid if they satisfy the initial constraints: $t \geq 0$ and $0 \leq u \leq 1$. The effective distance from the obstacle corresponds to the minimum t among the valid results. If the system has no solution, it means that the LiDAR ray and the obstacle edge segment are parallel. In this case, the LiDAR does not detect an intersection and proceeds to the next ray.

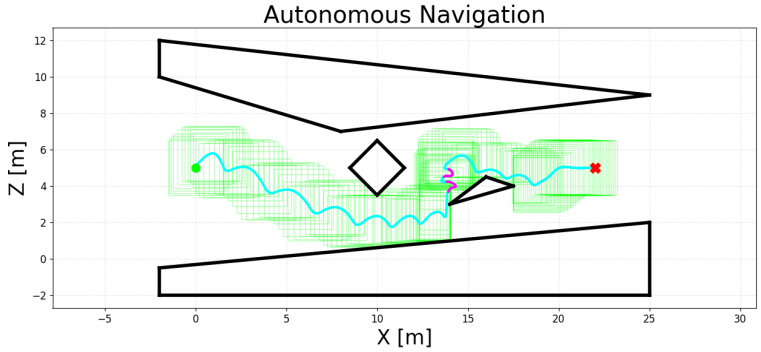


Figure 4.3: Drone trajectory and mapping in a more general enviroment

Note: The generalization of this concept to 3D space, which requires transitioning from line-segment to line-plane intersections, will be discussed in the dedicated chapter.

Chapter 5

Implementation, results and tests

This chapter presents the practical implementation and validation of the proposed methods. The analysis is divided into two macro-sections: the first part (Sections 5.1 and 5.2) illustrates the offline generation of the safe set RBOC and the training of the neural network; the second part (Sections 5.3, 5.4 and 5.5) evaluates the dynamic performance of the developed method (RBOC-MPC) through various navigation tests, progressing from simplified scenarios to more complex environments mapped via LiDAR.

The entire analysis presented in this chapter was first carried out for the simplified 2D version of the bicopter to limit computation times and facilitate the debugging phase, reducing the state dimension to $x = [p, v, \alpha] \in \mathbb{R}^7$. Then the code has been adapted for the 3D case of the quadcopter, which is discussed in another chapter.

5.1 Safe Set Generation via RBOC

To generate the training dataset for the Neural Network, RBOC (Eq. 3) was solved across a wide range of randomly sampled initial states.

Applying the iterative horizon expansion strategy, the distribution of the required horizons N to reach convergence was analyzed (Fig. 5.1). From it was possible to understand with which horizon N the solver was able to converge with an acceptable success rate η_{succ}, choosing N_{max} as the fixed horizon for the optimization (Eq. 3).

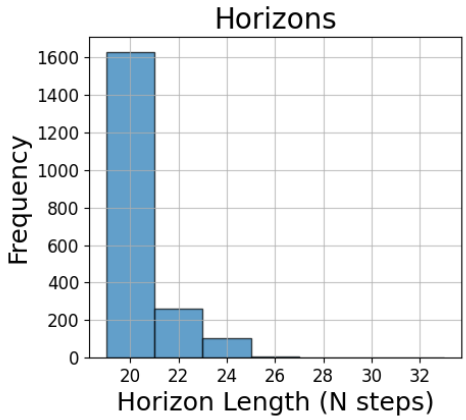


Figure 5.1: Distribution of different horizons in 2D

What was noted in this first phase is that to maintain a sufficiently high success rate

η_{succ}, a compromise must be reached between the allowable tolerance and the explored ranges for the initial conditions. In general, a larger tolerance allows sampling initial velocities and angles over wider ranges. A smaller tolerance, on the other hand, requires more limited initial ranges to achieve the same η_{succ}, while it also reduces the required maximum horizon N_{max} (Tab. 5.1).

Tolerance	**Velocity range [m/s]**	**Angle range [rad]**	$\eta_{succ}[\%]$	N_{max}
10^{-6}	± 1	$\pm\pi/2$	93.75	26
10^{-6}	± 1	$\pm\pi/3$	93	24
10^{-6}	± 1	$\pm\pi/4$	94	22
10^{-6}	± 0.8	$\pm\pi/2$	94.4	26
10^{-6}	± 0.8	$\pm\pi/3$	95.4	22
10^{-6}	± 0.8	$\pm\pi/4$	93.8	22
10^{-6}	± 0.7	$\pm\pi/4$	95.9	20
10^{-6}	± 0.6	$\pm\pi/2$	94.5	24
10^{-6}	± 0.6	$\pm\pi/4$	95.5	20
10^{-6}	± 0.5	$\pm\pi/4$	96.6	20
10^{-6}	± 0.5	$\pm\pi/2$	96.1	24
10^{-6}	± 0.5	$\pm\pi/3$	96.7	22

Table 5.1: Success rate analysis for minimum tollerance request for different initial conditions ranges on 1000 tests.

Having no particular constraints for the optimization horizon and wanting to be able to train the network on as many cases as possible, it was preferred to follow the first path, at the expense of a slightly higher tolerance (10^{-5})[1].

The distribution of the scaling parameter is shown in Fig. 5.2. Consistent with the theoretical formulation, since the shape vector $b_{sampled}$ is normalized, the scaling factor directly represents the physical dimensions of the safe-box in meters. It assumes reasonable values slightly larger than the drone's own footprint:

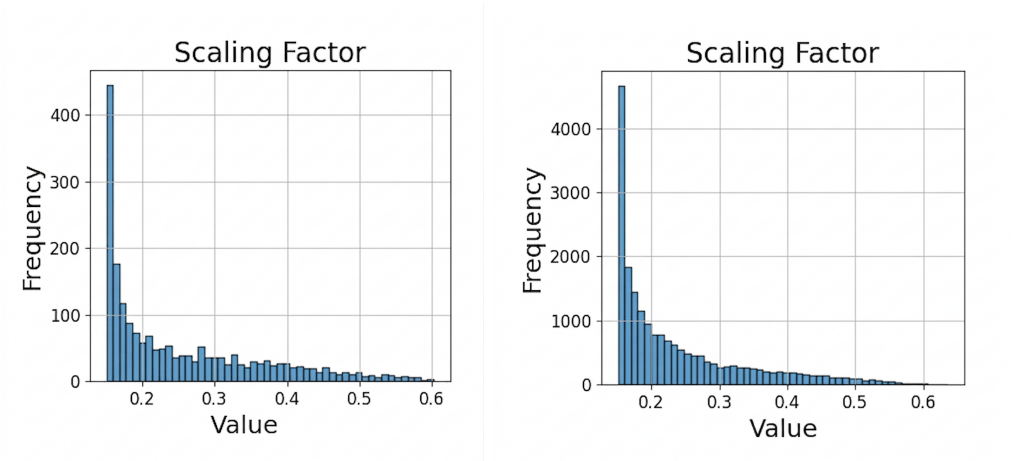


Figure 5.2: Distribution of Scaling Factor α in 2D with 2000 and 20000 samples

The results show that both the final velocities and the angle at the end of the maneuver

[1]Setting a tolerance $toll = 10^{-5}$ on the required value for the final state yields $\eta_{succ} = 99.95\%$ over 2000 tests and $\eta_{succ} = 99.90\%$ over 20000 tests.

are zero, as required. In the representation of the pose trajectory, in particular, the drone's footprint can also be observed, represented by the dashed blue line, while the dashed red line represents the dimensions of the box [2]. In particular, it is observed how the volume of the box never invades that of the drone, as required in the constraints (Eq. 4).

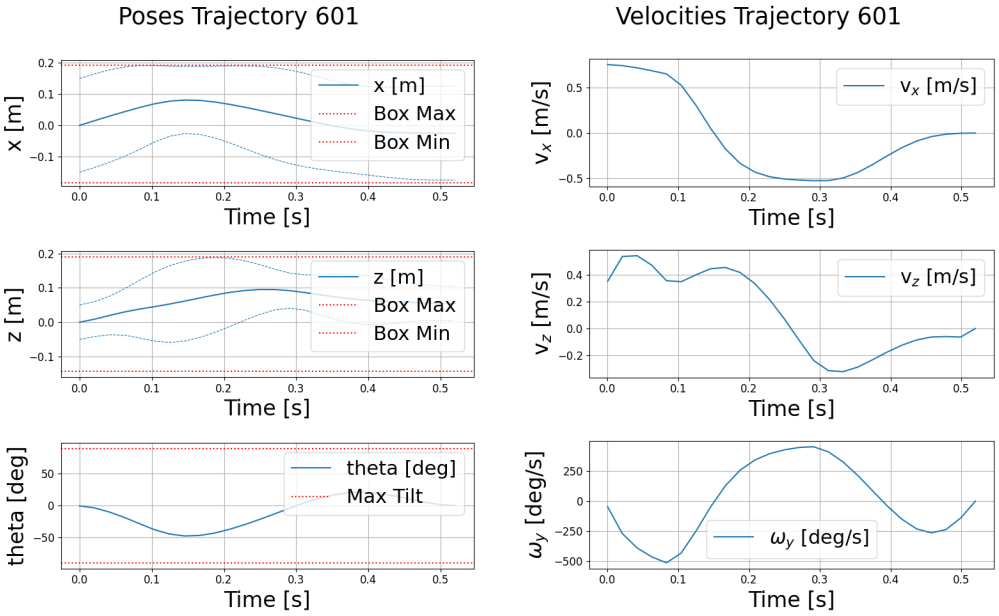


Figure 5.3: Results of OCP in 2D for one of the tests

[2]Having modeled the drone as an ellipsoid, the footprints along the two axes do not enlarge or shrink as it might seem; rather, it is an effect solely due to the rotation θ.

5.2 Neural Network training and validation

The Neural Network is trained taking as input the velocities and angles sampled in the dataset generated with RBOC. The training objective is to approximate the dimensions of the safe box within which the drone can successfully stop in stable conditions.

To evaluate the improvement in learning, the network was first trained with a more limited set (2000 samples) and subsequently with a larger set (20000 samples). As shown in Tab. 5.2, having more samples available evidently improves the learning process. Since the target is expressed in meters, the network's estimation error for the braking space is, on average, only 7 mm. Even in the worst-case scenario, the maximum absolute error remains within 15 cm, demonstrating a precision that is amply sufficient to guarantee the reliability and safety of the model in the performed simulations. The learning process proves effective even with limited datasets.

As illustrated in Fig. 5.4, the training and validation curves flatten rapidly and tend to overlap almost perfectly. This dynamic confirms that the network has developed excellent generalization capabilities on unseen data, successfully avoiding overfitting phenomena.

Samples	Data Set	Error	Value
2000	Model	RMSE	0.01083
		Max	0.25700
	Test	RMSE	0.01028
		Max	0.23750
20000	Model	RMSE	0.00743
		Max	0.17504
	Test	RMSE	0.00738
		Max	0.15277

Table 5.2: Final errors from the NN training with 2000 and 20000 samples

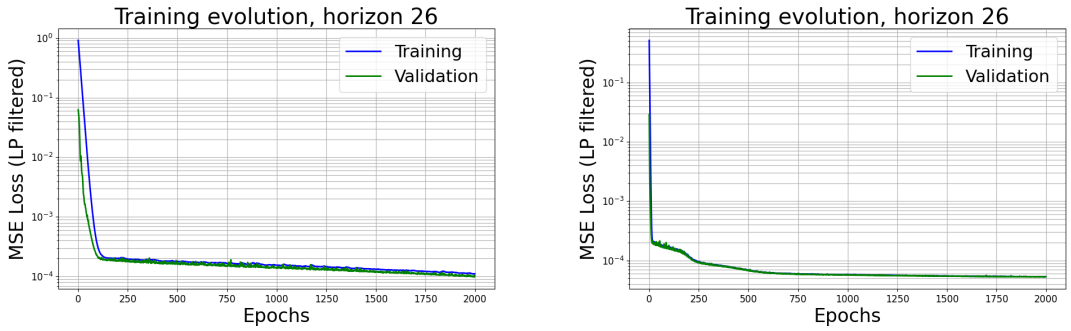


Figure 5.4: Training evolution in 2D with 2000 and 20000 samples

5.2.1 Safety Validation: Comparison with DeepReach

In order to further validate the robustness of RBOC, a direct comparison with DeepReach was conducted. The training and implementation of the DeepReach model for the drone dynamics under consideration were performed by Elias Fontanari [16]. The comparison focuses on pure predictive reliability: evaluating whether the two models can correctly deduce the survival of the drone starting from a known initial state.

To ensure a fair comparison, a new dataset was generated by solving an Optimal Control Problem (OCP) that serves as the Ground Truth for both networks. In this scenario, the OCP verifies that the drone remains within a randomly pre-sampled rectangular box for the entire simulation, starting from equally random initial conditions of angle and velocity. The predictive horizon used for the generation is $N = 25$, which, with a time step $dt = 20$ ms, corresponds to a simulated flight time of 0.5 s. Using DeepReach the avoid set for the correspondent problem in same settings was computed, in order to be able to predict if a state can stay in the given bounding box.

To test the Neural Network's generalization capability beyond the spatial limits seen during training, the test set was formulated in three variants, setting the randomizable maximum box dimensions to 1.0 m, 2.5 m, and 4.0 m, respectively. As the maximum dimensions of the box increase, the maneuvering space expands, and consequently, the success (feasibility) rate recorded by the OCP also grows. The predictive capabilities of the RBOC and DeepReach were evaluated using the following metrics:

- True alarm (TA): The model correctly predicts a failure confirmed by the OCP.
- False alarm (FA): The model predicts a failure, but the OCP finds a safe solution.
- True safe prediction (TSP): The model correctly predicts a safe state confirmed by the OCP.
- Missed alarm (MA): The model predicts that the state is safe, but the OCP fails. This represents the most critical error, as in a real-world application it would lead to a drone collision.

The two networks label a state as safe if the following constraints are met, respectively for the RBOC and DeepReach:

$$\begin{aligned} &\Phi\left(\phi, \dot{p}, \omega, \frac{b_p}{\|b_p\|}\right) \leq (1-\beta)\|b_p\| \\ &V\left(\phi, \dot{p}, \omega, b_p\right) > \alpha_{thr} \end{aligned} \tag{Eq. 1}$$

where $\alpha_{thr}, \beta \in [0, 1]$ are two parameters introduced to tune the conservativeness of the two methods. The trade-off trend for the medium-sized box (2.5 m) is presented in Fig. 5.5 to give an idea of the trends of the two curves.

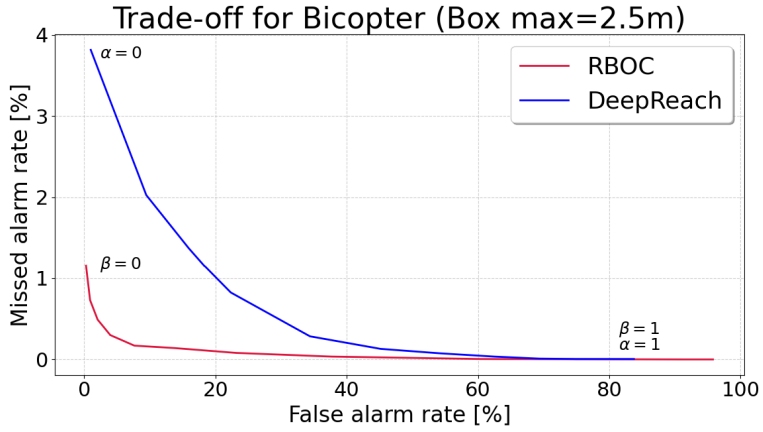


Figure 5.5: Performance trade-off between False Alarm Rate and Missed Alarm Rate for the Bicopter model

In the Bicopter case (Fig. 5.5) RBOC shows superior results because its curve lies strictly below the DeepReach curve across the entire analyzed spectrum. This means that RBOC consistently provides a lower rate of missed alarms (i.e. potential crashes) for the same false-alarm rate.

The results on 20000 samples for the case of zero conservativeness ($\alpha = \beta = 0$) are summarized in Tab. 5.3. As evident from the data, RBOC clearly outperforms DeepReach in the most critical metric. Although the increase in box dimensions inherently leads to fewer OCP failures, RBOC maintains a drastically lower number of Missed alarm (MA) compared to DeepReach across all three scenarios, demonstrating a significantly superior capacity to guarantee safety.

Range Box $0.15 - 1.0$ m OCP success rate 88.5%				Range Box $0.15 - 2.5$ m OCP success rate 95.8%				Range Box $0.15 - 4.0$ m OCP success rate 97.5%			
RBOC		DeepReach		RBOC		DeepReach		RBOC		DeepReach	
TA	1768	TA	229	TA	609	TA	77	TA	278	TA	42
FA	133	FA	250	FA	55	FA	199	FA	20	FA	155
TSP	17560	TSP	17443	TSP	19105	TSP	18961	TSP	19481	TSP	19346
MA	539	MA	2078	MA	231	MA	763	MA	221	MA	457

Table 5.3: Results of the comparison between the OCP and the two methods: RBOC and DeepReach

Moreover, regarding the computation times, RBOC turned out to be significantly lower than DeepReach (see Tab. 5.4). For RBOC, the time required for both generating the training set and training the Neural Network is taken into account.

Method	Space	Time
RBOC	2D	6 min
DeepReach	2D	20 h, 22 min

Table 5.4: Computation times of RBOC and DeepReach

5.3 Preliminary Evaluation in Global Path Planning (RBOC-MPC vs Std-MPC)

This section presents the results of the navigation tests conducted to validate the proposed framework (RBOC-MPC) versus the standard implementation (Std-MPC) without the NN terminal constraint. For all the simulations illustrated below, a global path planning approach is employed, assuming that the absolute position of the obstacle within the environment, x_{obs}, is known *a priori*. The discussion unfolds by first defining the simulation parameters, then analyzing in detail the system dynamics during a single flight, and finally validating the robustness of the algorithm through a statistical analysis across multiple randomized scenarios.

5.3.1 Test Parameters Definition

To evaluate the effectiveness of RBOC-MPC, a simple scenario was simulated featuring a single obstacle located at a known absolute position $x_{obs} = const$ in the World reference frame.

The drone starts from an equilibrium initial condition $x_0 = [0, 0, 0, 0, 0, 0]$ and is required to navigate close to the obstacle to reach an arbitrary target x_{target}, provided that the target state enforces zero velocities and angles, as learned by the Neural Network during training. An example of the resulting flight trajectory is illustrated in Fig. 5.6.

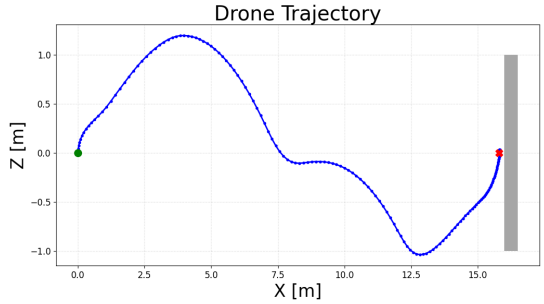


Figure 5.6: Example of drone trajectory

The primary comparison at this stage involves RBOC-MPC versus the Std-MPC. In this case and in the following tests the network was trained assuming that the box was a square, so the network does not need to take the parameter b as input.

For RBOC-MPC (Eq. 5), the terminal constraint has been defined as follows:

$$x_{obs} - x_N \geq \phi(\tilde{x}_N) \qquad \text{(Eq. 2)}$$

Conversely, in the Std-MPC case, there is no Neural Network, reducing the constraint to:

$$x_{obs} - x_N \geq 0 \qquad \text{(Eq. 3)}$$

To highlight the actual contribution provided by the Neural Network to the problem, the target was intentionally placed just before the obstacle at a distance $\Delta_{obs} = x_{obs} - x_{target}$. With large Δ_{obs} distances, both controllers successfully complete the maneuver. Indeed, the Std-MPC frequently exhibits an overshoot relative to the target, but there was the necessary physical space to recover and return to the target, as depicted in Fig. 5.7. However, by reducing Δ_{obs}, only RBOC-MPC managed to reach the target without colliding. Cases

in which the trajectory physically hits the wall are classified as failures in the subsequent statistical analysis.

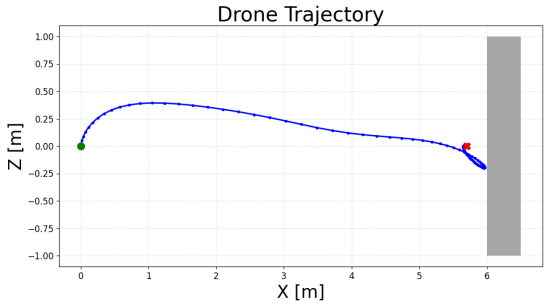


Figure 5.7: Drone trajectory with Std-MPC and large Δ_{obs} (motion inversion).

For the same reason, the target was placed at a significant distance from the starting point, allowing the drone to accumulate velocity during the initial flight phase. With short distances, conversely, the drone does not reach excessive speeds, rendering the two cases practically equivalent.

The simulated flight time T_{sim} is proportional to the distance traveled, as it is limited by the system's dynamics. Therefore, a sufficiently large value was also assigned to it.

5.3.2 Single flight test

Analyzing the behavior of RBOC-MPC over a short horizon ($N = 15$), it can be observed in detail from a single test in Fig. 5.8 that the drone successfully reaches the designated target, as shown in the pose and velocity plots. The velocities exhibit an oscillatory behavior while converging, due to the lower weights assigned to them in the cost function (Eq. 7). Analyzing the motor commands reveals an instantaneous control action at $t = 0$, with the motors starting from the hovering value. This behavior is justified by the modeling choice of excluding ground contact dynamics; therefore, the MPC must instantaneously apply thrust to compensate for gravitational acceleration and prevent free fall:

$$\begin{aligned} u_{warm_start} = u_{hover} &= \frac{mg}{2c_f} \quad \text{in 2D} \\ u_{warm_start} = u_{hover} &= \frac{mg}{4c_f} \quad \text{in 3D} \end{aligned} \tag{Eq. 4}$$

Where $c_f = 1.5 \cdot 10^{-3}$ is the thrust coefficient.

Finally, the last plot confirms that the drone never violates the safety constraint (Eq. 2).

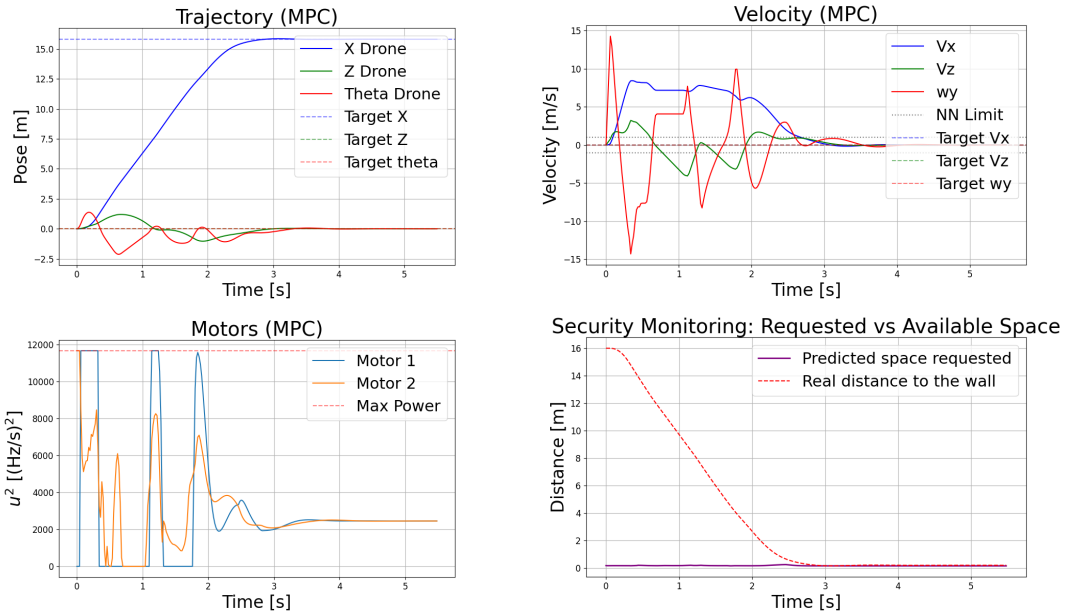


Figure 5.8: Single flight test with fixed obstacle and easy initial conditions.

In the Std-MPC (Eq. 3), when solving the exact same problem tested previously in Fig. 5.8, a critical failure of the optimizer occurs (Infeasibility, Status 4). The solver, having a prediction horizon that is too short ($N = 15$), detects the obstacle when the accumulated momentum is already too high (Fig. 5.10) to guarantee safe braking. Unable to find mathematically feasible solutions that respect the constraints, the algorithm fails before impact, leading to a potential loss of control of the real aircraft. The trajectory recorded by the drone in these cases does not represent a safe physical stop, but rather a solver stall: failing to compute any valid state values, it becomes unable to track a trajectory (Fig. 5.9). This scenario, along with the physical wall collision, is considered a failure in the subsequent statistical analysis.

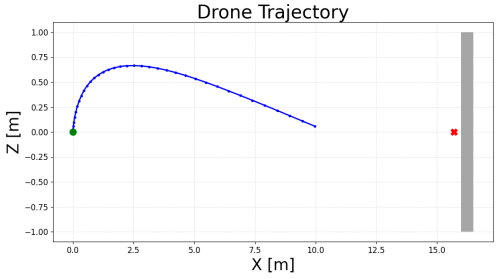


Figure 5.9: Incompleted drone trajectory with Std-MPC and critical conditions

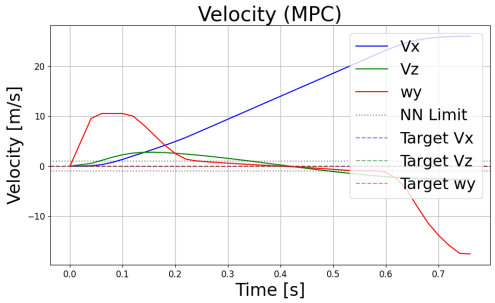


Figure 5.10: Drone velocities in Std-MPC with critical conditions

5.3.3 Multiple flight tests

By extending the analysis to a set of cases with varying initial conditions x_0 and distance margins Δ_{obs}, the effectiveness of RBOC-MPC compared to the Std-MPC approach was clearly demonstrated.

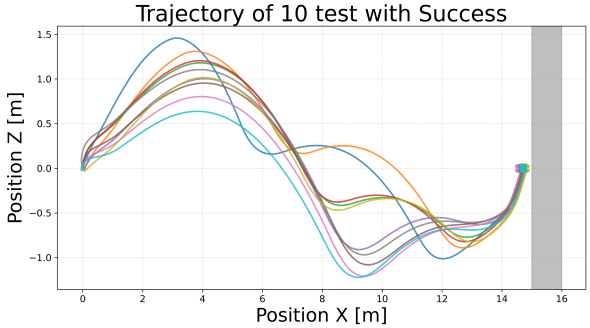


Figure 5.11: Multiple flight tests with different obstacle distance, targets and initial conditions.

As previously explained, if the simulation time T_{sim} is assumed to be constant and equal for all scenarios, it must be ensured that the tested cases involve physical distances that are traversable within that timeframe. This constraint becomes even more imperative when evaluating thousands of cases rather than a single instance. To avoid issues related to this factor, the simulation time T_{sim} is kept large in this analysis.

For the analysis, some parameters were allowed to vary randomly within an appropriate range (x_0, Δ_{obs}), while others were analyzed separately (x_{obs}, N).

The initial state x_0 was sampled both in the equilibrium state (null case) and in a broader range to demonstrate the behavior of the two approaches even under more critical initial conditions: $v_0 \in [-1, 1]$ m/s, $\theta_0 \in [-\pi/4, \pi/4]$ rad. As expected, success rates in critical conditions are lower than those starting from equilibrium.

The target was placed at varying distances to show that over long distances, the drone acquires too much speed to brake in time without the Neural Network's support. Regarding the distances between the target and the obstacle Δ_{obs}, the choice was dictated by the range defined in the safe set $\mathcal{V}$: approximately $\Delta_{obs} \in [0.2, 0.6]$ m (Fig. 5.2). Generally, for RBOC-MPC, selecting an interval above the upper limit implies certain feasibility. Conversely, dropping below the lower bound would prevent the drone from reaching the actual target, as the safety constraint imposed by the Neural Network would force the vehicle to stop before the obstacle. Therefore, it is more compelling to study the

MPC behavior strictly within this boundary interval. For the RBOC-MPC, it is expected that under bounded velocities (e.g., $v_0 \in [-1, 1]$ m/s, as per the computation of $\mathcal{V}$), the value ϕ predicted by the network will predominantly remain low (e.g. below 0.3), making it unlikely to exceed α_{real}. In other words, the probability of encountering a critical combination (critical state combined with reduced distance) that requires a value of ϕ higher than α_{real} (Eq. 2), is low.

The success rates of the two methods (RBOC-MPC and Std-MPC) are reported as a function of the obstacle distance from the null starting point ($d_{obs} = x_{obs} - x_0 = x_{obs}$), using a short horizon $N = 15$ and both equilibrium and critical initial conditions x_0 (Fig. 5.12). As previously mentioned, failures encompass both physical wall collisions and solver-declared infeasibility. In both initial conditions scenarios, the success rate of the Std-MPC dramatically decreases as the distance increases, confirming the findings from the single test. Conversely, the RBOC-MPC maintains an excellent success rate, showing only a slight decrease over distance when non-zero initial conditions are applied. Regardless of the scenario, the Std-MPC performs strictly worse than RBOC-MPC in terms of success percentages.

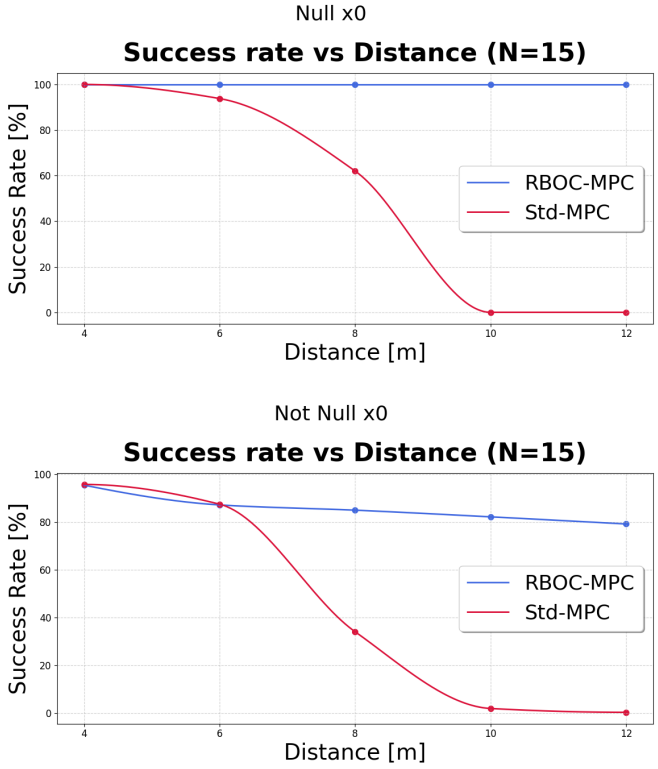


Figure 5.12: Success rate over obstacle distance for different I.C. on 400 tests

To conclude the analysis of the parameters affecting feasibility, the optimization horizon N must also be evaluated. All the limitations encountered so far between the two methods, both in the single test (Fig. 5.8 and Fig. 5.9, Fig. 5.10) and in the multiple tests (Fig. 5.12), hold true as long as the horizon N is kept short ($N = 15$). However, by extending the horizon length, the feasibility issue is automatically resolved (Fig. 5.13). Of course, the use of longer horizons increases computational times, which can make the approach incompatible with real-world deployments. As detailed in Tab. 5.5, doubling the horizon length from $N = 15$ to $N = 30$ roughly doubles the total computational time

required to solve the same number of simulated tests.

Problem	**N**	**Computational Time [min]**	
		$x_0 = 0$	$x_0 \neq 0$
Std-MPC	15	3.2	2.8
	30	6.9	6.4

Table 5.5: Computational Time of Std-MPC for $x_{obs} = 8$ m on 400 tests

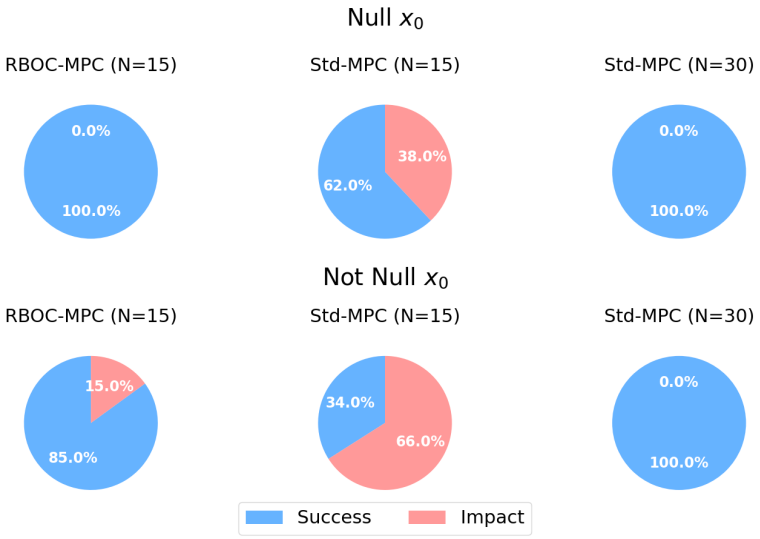


Figure 5.13: Success and Failure percentage for $x_{obs} = 8$ m on 400 tests.

5.4 Evaluation in Local Path Planning (LiDAR mapping tests)

This section presents the results of various tests conducted using the LiDAR mapping function, simulating a local path planning approach. As before, the simulations began with simpler environments before progressing to more complex scenarios. This approach revealed limitations in the mapping process, which was subsequently refined to enable the mapping of not only axis-aligned obstacles but also polygons of arbitrary shapes. Finally, the RBOC-MPC approach was compared with other obstacle avoidance methods—such as Std-MPC, Zero-TC MPC (Which corresponde to Std-MPC with null terminal constraints for angles and velocities) and APF—demonstrating superior performance.

5.4.1 Preliminary Tests on axis-aligned obstacles

Initial tests were conducted in three standard environments primarily to evaluate the correct operation of the mapping algorithm and the effectiveness of the RBOC-MPC in guiding the drone from a starting point to an assigned target without colliding with adjacent obstacles (Fig. 5.14). The first scenario features a wide corridor where the drone is expected to navigate freely, which is successfully demonstrated in the simulation. Conversely, it was observed that when facing frontal walls, the solver stalls, becoming trapped in local minima. A potential workaround for this issue could be slightly tilting the wall. However, the methods that have been developed so far for mapping are based on the perception of axis-aligned ("parallel") obstacles [11] [12] [14] [13] and would therefore require significant revision or complete modification to handle oblique obstacles. Consequently, this specific scenario has been omitted from the current study. On the other hand, the proposed solution performs optimally in the non-trivial case of two staggered walls. Here, it is clearly visible how the green safety boxes successfully extend towards the target, as intended by (Eq. 20).

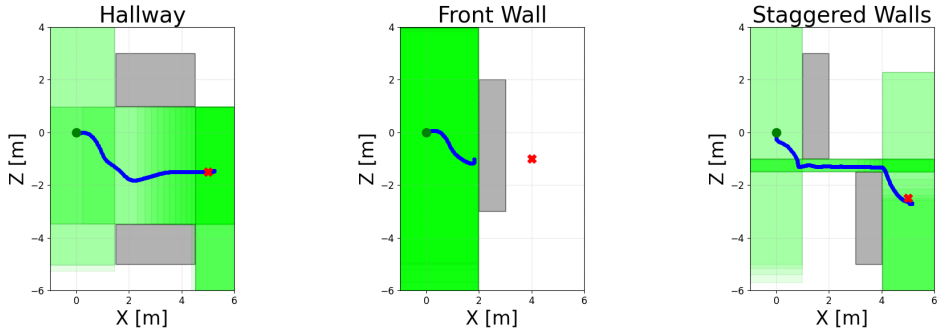


Figure 5.14: Tests on 3 different environments

5.4.2 Tests in more complex environments and problem analysis

Among the three navigation environments, the third is undoubtedly the most interesting. For this reason, the test was extended to more critical scenarios, where the drone must navigate through narrower corridors, performing constrained maneuvers and mapping significantly more obstacles simultaneously.

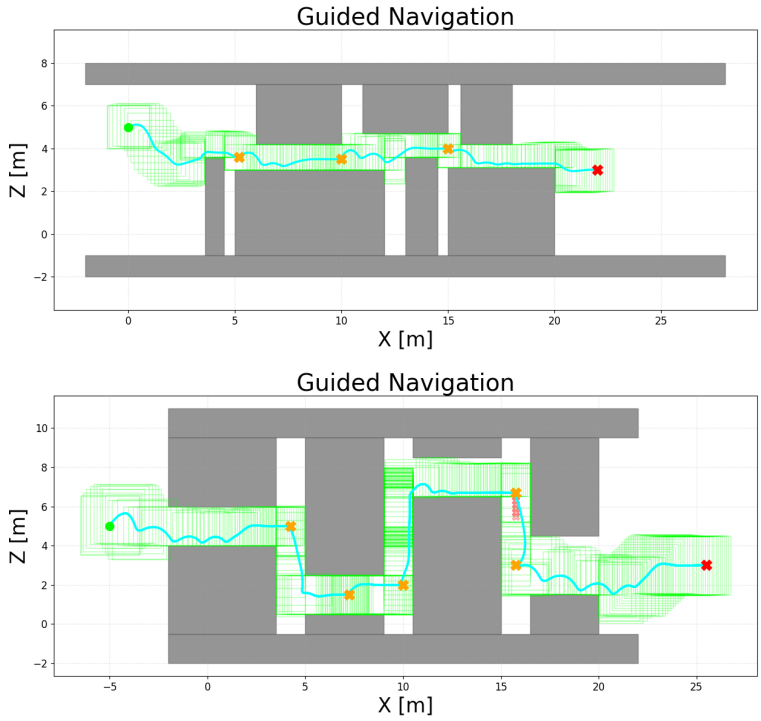


Figure 5.15: Drone guided navigation within a more critical environment

During this testing phase, some limitations of the method emerged, which were analyzed individually.

First, having so many closely spaced obstacles is not an optimal configuration for mapping, especially during the safe box extraction phase: the algorithm often continues to consider obstacle walls that have already been bypassed, restricting the drone's trajectory to spaces much smaller than what is actually available and preventing it from moving towards the target. For this reason, the maximum range of the LiDAR around the drone was limited (e.g. $1.5 - 2$ m), allowing it to map only the obstacles genuinely close to its current position.

Secondly, regarding obstacle management, it was observed that the maneuvers chosen by the drone sometimes lead to significantly different safety boxes from one step to the next (e.g., cornering maneuvers or high speeds along a straight segment). As long as the safe box instantaneously expands, there is no issue; however, if it shrinks, the drone may no longer be able to remain within the new box, and the solver records a critical failure (Infeasibility, Status 4). To overcome this issue, a drone rescue attempt is implemented, divided into multiple steps if the previous one fails:

- Recovery Mode Plan A: upon detecting a numerical failure of the solver, the system intervenes by resetting the internal memory of the MPC, and the previous warm-start trajectory is replaced with an assumption of spatial stationarity for the control inputs (hovering);
- Recovery Mode Plan B: if plan A fails, the drone is guided to the center of the box calculated a few iterations prior (e.g. 10 steps back)[3] via a Std-MPC with final targets set to zero for both velocities and angles.

[3]The center of the box was chosen as a safe point because it is the furthest from all obstacles. The center of a previous box is used to avoid having a destination point too close to the current state, giving the drone the opportunity to adjust its velocities and approach the target values with

Although modifying the target strictly affects the cost function rather than the collision avoidance constraints, switching to a Std-MPC eliminates the terminal constraint imposed by the Neural Network, practically relaxing the mathematical bounds of the problem. Furthermore, shifting the target alters the cost function gradients, helping the solver escape the locally ill-conditioned areas where it previously stalled.

Each of these procedures provides the RBOC-MPC with a numerically safe starting point, allowing the solver to recalculate from scratch an optimal and safe trajectory adapted to the new configuration of the surrounding free space. With this method, the drone can complete trajectories with greater autonomy, mapping even larger areas with the LiDAR, and crucially, managing potential infeasibility conditions.

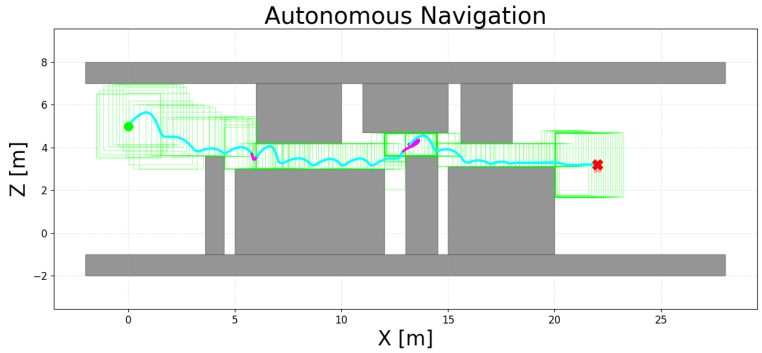


Figure 5.16: Drone autonomous navigation with recovery highlighted in purple

Finally, the last limitation concerns the definition of the path to follow. Since the algorithm was not developed for global planning and relies solely on local path perception provided by the LiDAR, this issue is considered secondary compared to safety management. The simplest approach is to guide the drone step-by-step, having it follow sequential waypoints (Fig. 5.15). However, this choice is trivial and prevents observing the drone's true behavior in more critical environments and situations. If, conversely, the drone is left completely free with a single, very distant target at the end of the room, it can be observed that it might encounter infeasibility conditions (Fig. 5.16) (which we have already solved with the aforementioned methods), or stall conditions. The latter problem occurs, for instance, if the drone is vertically aligned with the final target and therefore dedicates all its effort to expanding the safe box horizontally. As long as there is room to move horizontally, this maneuver is valid; but if, for example, it faces a wall, it will remain stuck at that point (local minimum) for multiple solver iterations, neither completing the path nor returning an infeasibility condition (Fig. 5.17).

a less critical maneuver.

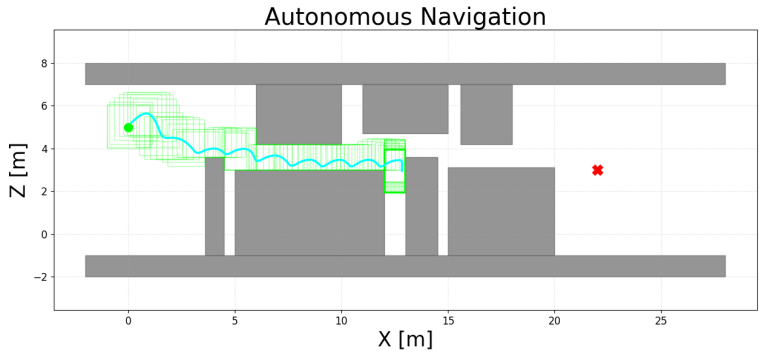


Figure 5.17: Drone autonomous navigation not completed due to stall

It was observed that a quick solution can be obtained by better calibrating the target position, precisely to prevent the solver from getting stuck along the walls (Fig. 5.18).

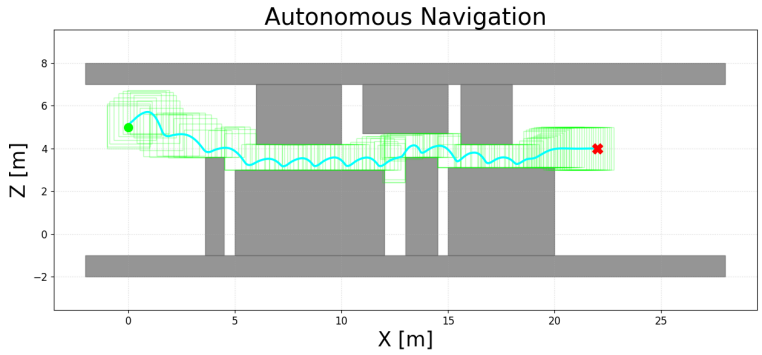


Figure 5.18: Drone autonomous navigation completed for target recalibration

Inherently, establishing the suitable position to indicate the path the drone should take is not a simple, nor a general, solution. Therefore, as an initial approach, a stall recovery method for the drone was defined. It is assumed that the final target or the intermediate waypoint is located in an obstacle-free space, at least along the vertical direction. If the solver detects that the drone has not made significant movement for several iterations, it shifts the target higher by modifying the Z coordinate. The choice to move the target higher is based on the premise that the issue lies in overcoming frontal walls; thus, bringing the target closer or further along X would not improve the situation. To bypass the obstacle, the target must therefore be moved vertically: assuming any environment has a floor, moving the target downwards is more limiting than moving it upwards, and real-world obstacles rarely descend entirely from the ceiling. For all these reasons, the target is shifted vertically upwards in case of a stall. Should an environment have obstacles that must be bypassed from below, a waypoint would be necessary to direct the drone in that direction. Repeating the exact previous simulation where the stall occurred (Fig. 5.17), it can be seen how the target autonomously moves into the same configuration for which the trajectory was coincidentally completed earlier (Fig. 5.18). This makes the method autonomous in managing stalls.

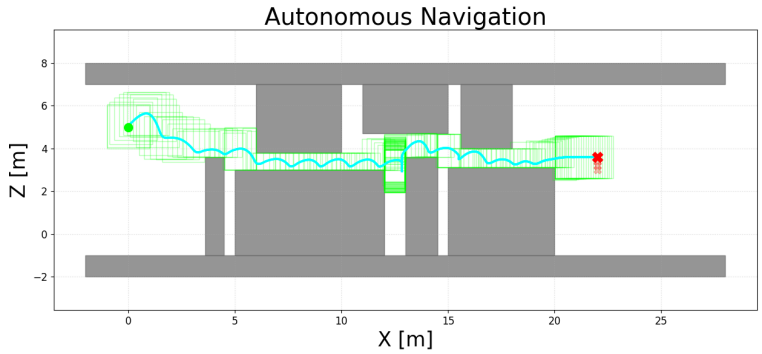


Figure 5.19: Drone Autonomous navigation completed with stall management

Improvements in infeasibility management

The previous recovery Plan B (now declassified to Plan C) significantly increases simulation times when activated. Due to the lack of global planning, the drone tends to fall back into the same errors despite restarting from scratch. This approach causes it to enter a very long, computationally demanding, and sometimes infinite loop, which is highly suboptimal for the solver's performance.

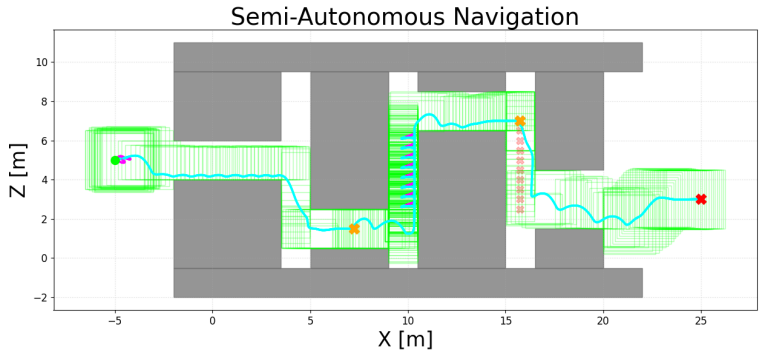


Figure 5.20: Drone Semi-Autonomous Navigation completed with numerous recovery cycles

For this reason, a third, more classical recovery solution was introduced, to be undertaken after utilizing hovering for the motors and before redirecting to the center of the box. The new Plan B consists of updating the warm-start of the control inputs with the values recorded at previous steps (Historical Warm-Start), starting from the one immediately preceding the failure and repeating the optimization. Should it fail, the solver attempts the resolution again by initializing the motors with the step prior to that, and so on, back to the beginning of the simulation[4]. Only if all motor values prove unsuccessful does the recovery management pass to Plan C, namely the return to the center of the box. If, instead, one of the past control inputs produces a safe solution, the solver exits the recovery mode and resumes normal operation.

This new solution partially reduces simulation times by allowing the solver to test dynamically coherent starting points derived from past motor values.

[4]To further speed up the solver, for example in a real-world application rather than a simulation, instead of rolling the warm-start backward to the very beginning of the simulation, the backward steps can be limited to a fixed number (a "recent history of steps").

Parameters Recalibration

It was observed that a too low number of iterations does not guarantee that the solver reaches the convergence condition. With few iterations, the result often coincides with a sub-optimal convergence, causing the drone to generate physically impossible trajectories (e.g., passing through obstacles). For this reason, the number of iterations was kept high both for non-linear-problems and quadratic-problems[5].

5.4.3 Analysis on Multiple Cases for generalized mapping

The mapping implemented for "parallel" obstacles has been extended to obstacles of arbitrary shape and to confirm the validity of generalized mapping method, a test environment containing various generally shaped obstacles was created. All obstacles can be overcome from above due to the logic underlying the stall management. Several targets, which the drone must reach autonomously, are then placed on the map to cover its most significant points.

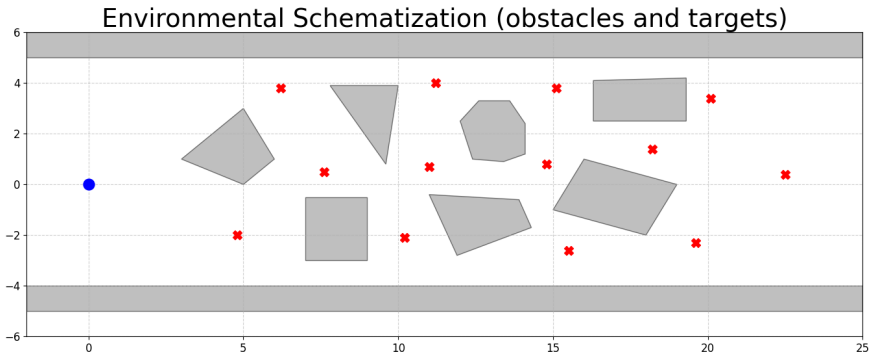


Figure 5.21: Test environment

The comparison is carried out between the case of RBOC-MPC and the Std-MPC, observing the success and failure rates as the most significant parameters vary.

What has been observed after several tests is that the most significant parameters for the success of the various paths are the LiDAR range and the prediction horizon N. In particular, excessively wide ranges, as already anticipated, lead the drone to incorrectly estimate the safe-box in which it can move: the success rate in both RBOC-MPC case and Std-MPC case decreases as the LiDAR range increases.

[5]Max iterations: $nlp = 100$, $qp = 100$.

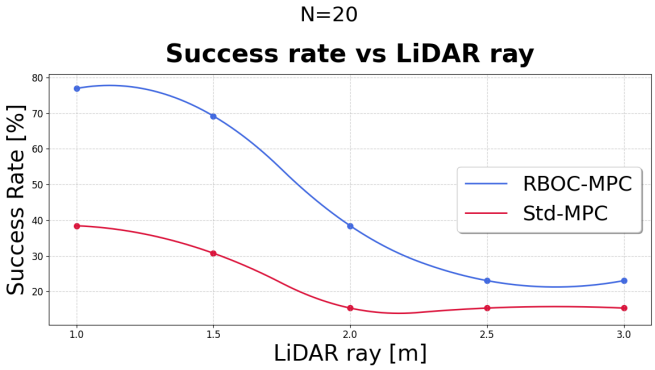


Figure 5.22: Success rate analysis based on LiDAR ray with fixed horizon

Please note that three different situations were considered for the analysis:

- success: the drone actually reaches the desired target
- timeout: the drone, while remaining in a safe condition, stalls at a point on the trajectory (local minimum), exhausting the iterations before reaching the target
- crash: the drone enters a critical condition (status 4) that cannot be resolved with any of the available recoveries, resulting in a failure

The timeout case is to be considered a success for safety management, as the drone does not record an actual crash (infeasibility), but if we look at the achievement of the target it is a failure towards the final objective.

Conversely, excessively short horizons tend to increase infeasibility conditions (Status 4), raising the probability of trajectory failure. Generally, increasing the horizon increases the success rate. However, the results show that with a limited range and a very long horizon, the effective success rate tends to drop again in the RBOC-MPC case, while it remains constant (saturation) in the Std-MPC case. Having a very long horizon with a LiDAR whose field of view is limited might not make much sense, as it would cause the drone to calculate a trajectory for a future instant that falls outside the actual mapping view.

Note: It should be noted that in the analysis of the Std-MPC case, some tests show false successes: the drone actually reaches the target, but by taking an incorrect trajectory and partially penetrating the obstacles due to the inability to stop in time. For this reason, in the Std-MPC case, all quantitative results regarding success percentages were subsequently validated through graphical representation, and only genuine, physically feasible successes were considered.

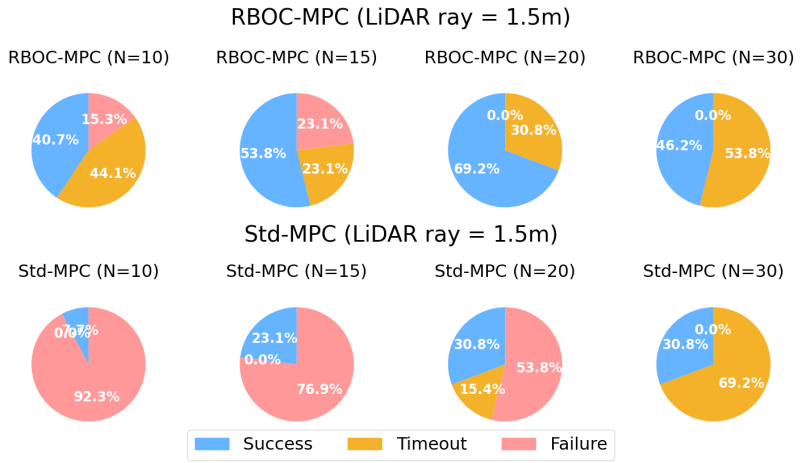


Figure 5.23: Success rate analysis based on horizon N with fixed LiDAR ray

Study of better techniques for the definition of the safe-box

With the safe-box definition logic tested so far (Eq. 20), the highest achievable success rate is close to 80%, but only by strictly limiting the LiDAR range (Fig. 5.22). For this reason, alternative strategies for defining the safe-box were tested.

In general, a limitation of the approach used so far (Case 1 or Baseline Case) is that it establishes the safe-box by rewarding extension toward the final target, which is typically a distant point and does not necessarily lead to the creation of optimal spaces for the drone to move in. To address this, an alternative strategy utilizes the trajectory predicted by the MPC itself. The directional vector is calculated as the difference between the predicted position and the current position of the system $v_{dir} = x_{pred} - x_{current}$ (Eq. 19). Case 2 evaluates the use of the state prediction vector v_{dir} to fix the dimensions of the box in two directions: the projections of the vector itself ($v_{dir,x}$ and $v_{dir,z}$ in 2D) are used as limits for two faces of the safe-box. The remaining two unconstrained faces are left free in order to maximize the area. The results, however, showed a minimal success rate. This approach likely creates overly rigid constraints for the system, whether the number of steps n is comparable to the horizon N or if it is smaller ($N > n$), frequently leading the solver to infeasibility (Status 4).

To mitigate the rigidity of the previous approach, Case 3' initializes only the two faces of the box with the prediction vector, leaving them free to adapt during area maximization. However, simulation results indicate that this method fails with exactly the same percentage as the first one, primarily due to the geometric limitation of pure area maximization: when required to exclude an obstacle by sacrificing the fewest possible square meters, the function tends to further shorten the front face (already initialized very close to the drone) rather than cutting the large rear or lateral faces. The result is the complete occlusion of the direction of motion.

The main reason these two new approaches fail is that the logic implemented and used so far was incorrectly applied; this logic expects to start with large boxes during initialization that need to be reduced. Cases 2 and 3', instead, start with small boxes that are not enlarged during the area adaptation phase and are also insufficient for the drone's movement. This aspect of area reduction is somewhat mitigated in Case 2, which ensures no cuts are made along the direction of motion, but it is completely absent in Case 3'. For this reason, a new expansion function was created that, starting from these minimal

boxes, pushes the box to expand, while still excluding adjacent obstacles. Based on this explanation, this expansion logic is tested on the previously flawed Case 3', which becomes the actual Case 3 under analysis. Although better than the previous attempt (Case 3'), the new approach does not have a high success rate, and the reason might be related to the fact that the face expansion is generic and non-directional.

In fact, all the new approaches tested thus far focused on maximizing the area, but without providing any directional guidance for the created safe-box. To provide directional guidance, Case 4 utilizes the prediction vector v_{dir} to direct the box expansion from the previous Case 3: the faces of the box are expanded starting from the direction the drone is facing. This attempt does not yield a higher number of successes but reduces crashes by increasing the number of timeouts. So it was decided to partially revert to the previous method (Eq. 20), which yielded better results. What changed was the target assigned to the function to define the bonus, along with the weight W, which was recalibrated ($W = 50$).

To avoid relying on past predictions (Eq. 19), Case 5 explores the use of the current state x_t as a local target to adapt the box, specifically utilizing the velocity vector v_x and v_z. By using these values as local targets to adapt the box, the results are comparable to the previous ones, although they remained inferior to the initial baseline case.

Finally, Case 6 combines the prediction vector (Eq. 19) as a local target with the bonus logic (Eq. 20). This final formulation proved to be the most successful among all those tested and the trajectories performed are depicted in Fig. 5.24.

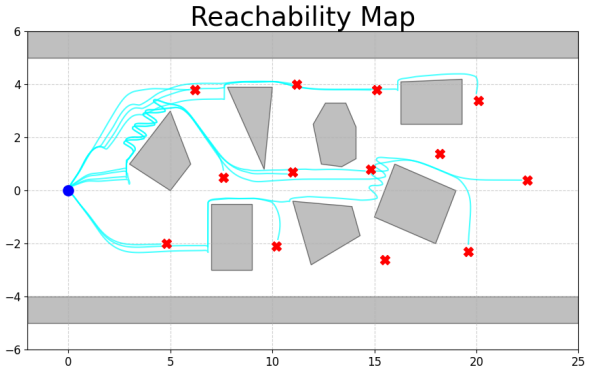


Figure 5.24: Drone trajectories for the actual best mapping method (Case 6)

Summarizing the various approaches tested for the creation of the safe-box:

- Case 1: area maximization and bonus toward the final target (Baseline Case)
- Case 2: projections of the predicted trajectory vector for initialization (locked faces) and area maximization
- Case 3: projections of the predicted trajectory vector for initialization (free faces) and area maximization with subsequent generic expansion logic
- Case 4: projections of the predicted trajectory vector for initialization (free faces) and area maximization with subsequent directional expansion logic
- Case 5: area maximization and bonus toward a local target defined by the current velocity vector

- Case 6: area maximization and bonus toward a local target defined by the predicted trajectory vector

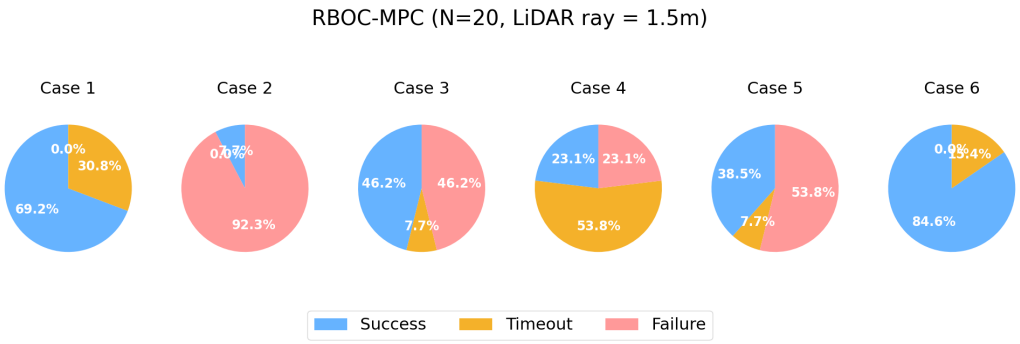


Figure 5.25: Success rate analysis for different mapping cases

Note: Even in the best case (Case 6) the LiDAR range is kept limited because, as previously highlighted (Fig. 5.22), with larger ranges it has been observed that obstacles are handled inadequately and safe boxes are created incorrectly, making successes purely random events (success rate drops from 84.6% to 23.1%).

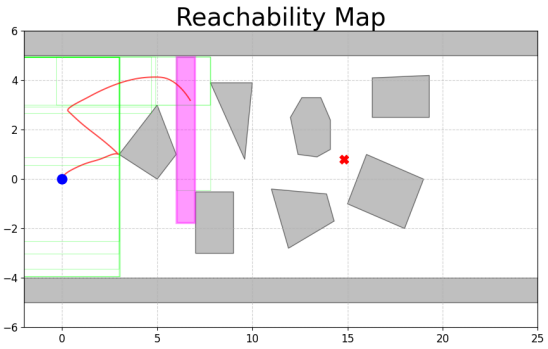


Figure 5.26: Wrong box created for LiDAR radius too large

In-depth analysis on failures and introduction of Warm Start Method

RECURSIVE FEASIBILITY An in-depth analysis of the failures suggests that they are associated with the predicted trajectory exiting the imposed safe-box.

Knowing that the safe-box is inherently a constraint for the MPC, the solver will always compute a trajectory at step k that remains within that box at the same iteration, without needing to impose it as an additional check during the box construction. When the solver fails and encounters an unresolvable status 4, one of the causes can indeed be that the constraint of staying inside the safe-box is no longer respected. The probability of the predicted trajectory remaining inside the box (and thus whether it fails or not) depends on the horizon N and the maximum limit of the box: if N is small, the trajectory is more easily contained even by small boxes, but if N is large, the predicted trajectory will be forced to fold back to fit inside the box if the latter is small.

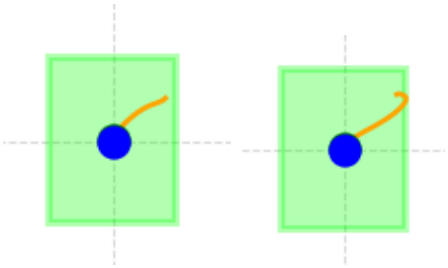


Figure 5.27: Containment of the predicted trajectory in the safe box with small and large horizon

With larger horizons, given the same box dimensions, it is therefore easier for the constraint to be violated, thus registering a status 4. This explains the anomalous behavior of the drop in successes with large horizons recorded previously (Fig. 5.23).

However, it is not guaranteed that the trajectory predicted at step k will be contained in the safe-box of the subsequent iteration. To verify this specific violation, the failure states were examined, revealing that all critical failures register the predicted trajectory exiting the box near the iteration of the critical failure. This leads us to conclude that there is a strong correlation between the two events, and the exit of the previously predicted trajectory from the safe-box is likely a sufficient condition for the drone to crash. To address this limitation, the previous approaches (Case 3 and Case 4) were modified by initializing the box not just with a few steps of the previously predicted trajectory $traj_{k-1}$, but by ensuring that it contains all of it: as explained earlier, the safe box_{k-1} is used as a warm start for the next iteration. Applying this warm start technique to the safe-box and testing both the generic and directional expansions of the previous Cases 3 and 4, it was observed that the results are identical in both cases: the success rate drops in Case 3 while remaining the same in Case 4, but failures become zero as they are replaced by timeouts, and no recovery is triggered:

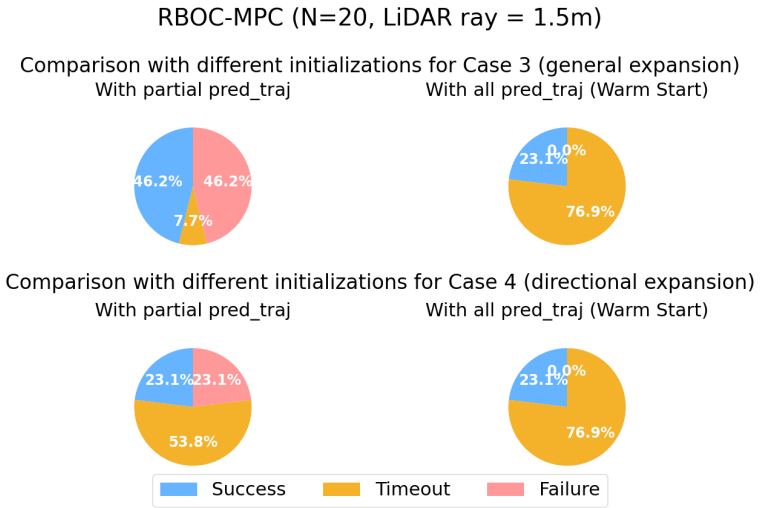


Figure 5.28: Improving previous methods (Case 3 and Case 4) with the application of warm start

MAPPING INEFFICIENCY Undoubtedly, this warm start approach for the safe-box makes the path taken by the drone safer, as it chooses more conservative trajectories,

eliminating crash conditions. However, these tests make the inefficiency in path planning even more evident. Previously, the problem was related to the fact that often, despite having room to move, the drone predicted trajectories that brought it close to obstacles and consequently to critical status 4 conditions. This behavior was present regardless of how the safe-box was computed in the different tested cases (Fig. 5.25) (e.g., Case 6: actual best case vs Case 4). Now, having a more conservative approach, this specific problem does not occur, but by staying far from obstacles, the drone often requires more space to perform a maneuver, and in tight directional changes, this sometimes leads it to take wrong paths. For example, during a vertical climb to perform a 90° turn, the drone must rise well above the upper limit of the obstacle to convert the vertical box into a horizontal one and thus execute the turn. If there is not enough space (e.g., due to an upper ceiling limit), the drone remains blocked.

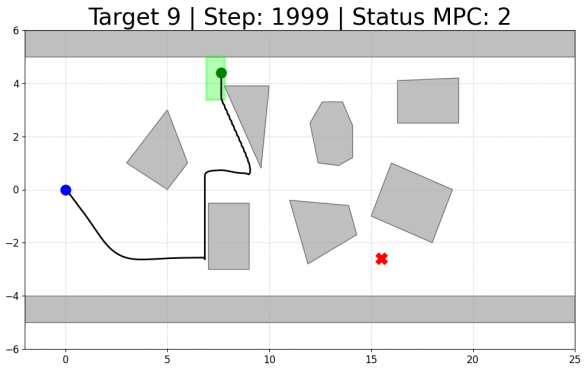


Figure 5.29: Example of drone trajectory in the Warm Start scenario, leading to a timeout

Certainly, the various methods for creating the safe-box influence the trajectory calculation, but at the same time, the trajectory has consequences for the creation of the boxes themselves. In other words, making the drone take different paths results in different safe boxes and vice versa, causing it to encounter more or less critical situations, which fail at different rates.

It should be noted, however, that the optimization is not based on calculating the best path to bypass obstacles, but rather the path to reach the target while ensuring it does not hit any obstacles. The underlying problem is probably exactly how the drone predicts the trajectory: its only objective is to reach the target, and it has no idea which path to follow; it will proceed toward that point, and upon finding an obstacle in front of it, this will only act as a constraint, without calculating the best way to bypass it. This behavior is seen, for example, when the drone, upon reaching a point in the path, continues to relentlessly push against the obstacle, remaining stalled where it is and moving at most a few centimeters, causing the simulation to end in that state due to the exhaustion of iterations. The problem is apparently linked precisely to this aspect and the fact that the drone does not know where to go (lack of global planning), rather than on safety management. If larger horizons were used, the solver would probably manage trajectories better by bypassing obstacles, but doing so would lose the effect of the Neural Network compared to the Std-MPC case and, as previously stated, larger safe boxes would be necessary to contain the trajectory without distorting it. Doing so would reintroduce the frequently explained problem (Fig. 5.26) where a large LiDAR radius creates sub-optimal boxes.

5.5 Validation of definitive mapping method

The extensive testing campaign conducted in the previous section highlighted that the safe-box generation entails a delicate trade-off between the spatial efficiency of the trajectory and the safety of the vehicle. Although some approaches (such as Case 6) showed encouraging success rates, their inherent instability under varying operational parameters, coupled with the occurrence of critical collisions, makes them unsuitable for real-world applications.

Therefore, establishing safety and the absolute guarantee of collision avoidance as the fundamental priorities of the system, the choice for the definitive mapping method fell on the Warm Start approach. Following numerous analyses and calibrations, this initially overly conservative method was refined to mitigate its navigation inefficiencies, leading to a robust final formulation.

This section first illustrates the limitations of Case 6 in comparison with the results obtained from the definitive box extraction model, followed by a quantitative performance comparison between RBOC-MPC and other well-known state-of-the-art approaches for obstacle avoidance.

5.5.1 Instability of Case 6 and improvement of the Warm Start method

Analyzing the behavior of the current best case (Case 6: restriction of the large box focusing on the local target given by the prediction) more closely, it is evident that the results change significantly as the horizon and the LiDAR range vary. Although the tests were conducted on a few samples, they give an idea of the method's instability. However, it is confirmed that with small safe-boxes, increasing the horizon decreases performance, while with larger boxes, increasing the horizon does not change the success rates, which tend to saturate. Timeouts and failures, on the other hand, show a more fluctuating trend, and it can be seen that with large horizons timeouts tend to increase while failures decrease. Nevertheless, from what has been observed in these tests (Fig. 5.30), a general trend cannot be established, and the behavior of this method remains somewhat random.

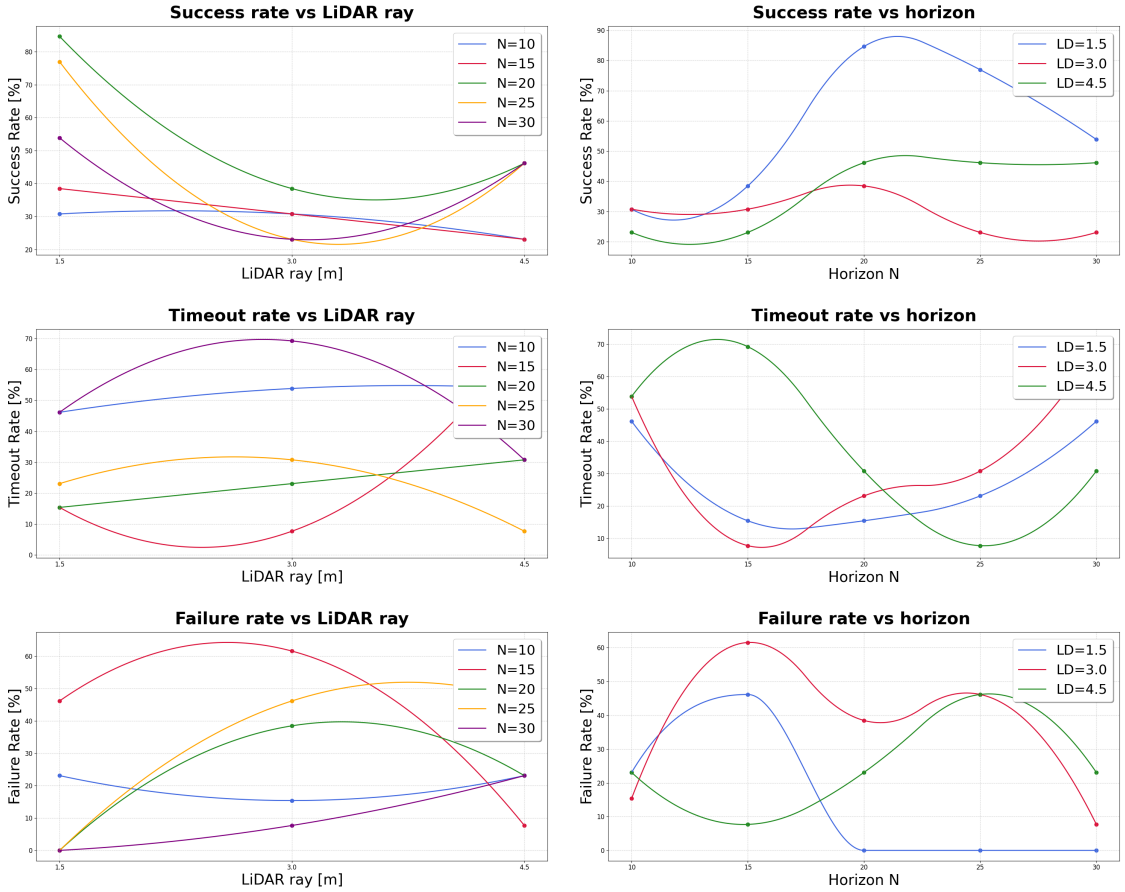


Figure 5.30: Performance metrics as functions of LiDAR range and horizon length for Case 6.

Choosing precisely to prioritize safety by minimizing failures, the right approach is the Warm Start (Fig. 5.28), which guarantees recursive feasibility. However, as it has been implemented so far, the approach has proven to be too conservative, taking too much time and too many iterations during directional changes (Fig. 5.29). This problem is solved by not using the entire safe-box defined in the previous iteration, but rather by applying the refined extraction method formalized in Sec. 4.4.2, which initializes the smallest possible safe-box around the entire predicted trajectory, guaranteeing the safety margin M.

Although no significant differences were observed between the directional and generic expansion of the box using the Warm Start (Fig. 5.28), intuitively, the former, if well managed, can yield better results by reducing failures (Fig. 5.25). By guiding the extension through the directional priority criteria outlined in Sec. 4.4.2 (based on the local prediction vector and the final absolute target), timeouts can indeed be reduced, preventing the drone from taking long, inefficient paths or getting stuck because the safe-box does not expand correctly. It has been observed that for small LiDAR ranges, this distinction is almost irrelevant, but it becomes significant for larger ranges where often expanding one face in one direction first limited the extension in the other due to the presence of multiple obstacles, forcing the drone to remain blocked due to a wrong choice in defining the safe-box. For the priority among the various axes, it was decided to follow the final target because it is less variable than the local one, always considering the previously highlighted double dependency between the creation of the box and the trajectory itself.

The simulation results demonstrate that short horizons yield a higher number of suc-

cesses but create less linear trajectories. Larger horizons lower the success percentage, as does increasing the LiDAR range, exactly as seen in all previous tests. Having large horizons causes the MPC to predict a longer trajectory, initializing a safe-box with larger dimensions and thus limiting its expansion. Having a LiDAR with an overly wide range, as already mentioned, causes the drone to perceive many more obstacles. Furthermore, very small horizons increase the number of triggered recoveries. Inevitably, despite the various route guidance strategies, there are still cases where the drone fails to reach the target, remaining stuck in erroneous loops. This behavior is not anomalous given what was previously mentioned regarding the inefficiency of mapping and local path planning.

The results obtained with this method in the same environment as the previously tested case (Fig. 5.30) are reported below:

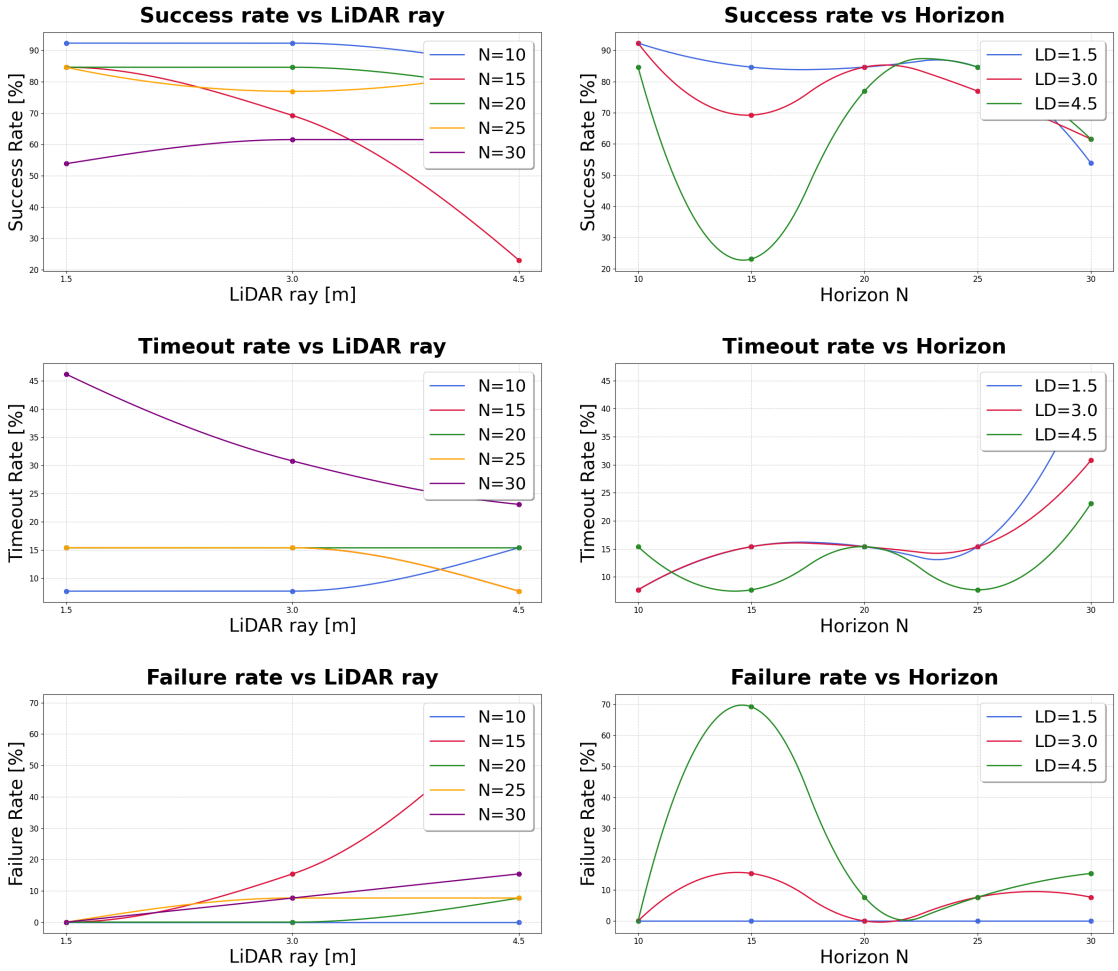


Figure 5.31: Resulting percentages as a function of LiDAR range and horizon for modified Warm Start

In-depth analysis of the safety margin

An evaluation of different safety margins M reveals that the results can vary slightly depending on the chosen value. Since the choice of the margin is arbitrary and the tests performed are limited to a specific environment, there is no single margin value that can be considered optimal. The results obtained for different LiDAR ranges and horizons with different margins are reported below:

Margin 0 **cm**				
LiDAR ray	**N**	**Success %**	**Timeout %**	**Failure %**
1.5	10	92.31	7.69	0
	15	84.62	15.38	0
	20	69.23	30.77	0
	25	84.62	15.38	0
	30	61.54	38.46	0
3.0	10	84.62	7.69	7.69
	15	61.54	0	38.46
	20	69.23	0	30.77
	25	69.23	23.08	7.69
	30	69.23	30.77	0
4.5	10	53.85	30.77	15.38
	15	7.69	0	93.31
	20	38.46	0	61.54
	25	69.24	15.38	15.38
	30	53.85	30.77	15.38

Table 5.6: Resulting percentages for the Warm-Start method with a null margin

Margin 15 **cm**				
LiDAR ray	**N**	**Success %**	**Timeout %**	**Failure %**
1.5	10	92.31	7.69	0
	15	84.62	15.38	0
	20	84.62	15.38	0
	25	84.62	15.38	0
	30	53.85	46.15	0
3.0	10	92.31	7.69	0
	15	69.24	15.38	15.38
	20	84.62	15.38	0
	25	76.93	15.38	7.69
	30	61.54	30.77	7.69
4.5	10	84.62	15.38	0
	15	23.08	7.69	69.23
	20	76.93	15.38	7.69
	25	84.62	7.69	7.69
	30	61.54	23.08	15.38

Table 5.7: Resulting percentages for the Warm-Start method with a 15 cm margin

Margin 20 **cm**				
LiDAR ray	**N**	**Success %**	**Timeout %**	**Failure %**
1.5	10	61.54	23.08	15.38
	15	84.62	15.38	0
	20	84.62	15.38	0
	25	84.62	15.38	0
	30	53.85	38.46	7.69
3.0	10	100	0	0
	15	84.62	15.38	0
	20	69.23	7.69	23.08
	25	84.62	15.38	0
	30	53.85	15.38	30.77
4.5	10	92.31	0	7.69
	15	84.62	15.38	0
	20	76.93	15.38	7.69
	25	69.24	15.38	15.38
	30	61.54	23.08	15.38

Table 5.8: Resulting percentages for the Warm-Start method with a 20 cm margin

With limited LiDAR ranges (e.g., 1.5 m), changing the margin does not seem to have major consequences on the results, except for very short horizons where there is a clear worsening as the margin increases. With wider LiDAR ranges (e.g., $3.0-4.5$ m), the larger margin (Tab. 5.8) produces slightly better results than the narrower one (Tab. 5.7), and much better results than the zero-margin case Tab. 5.6). From these observations, it can be hypothesized that the margin must assume a specific value depending on the LiDAR range and the size of the safe-box to bring a concrete improvement to the method.

Evaluation of computational times

To determine the optimal horizon for further testing, computational times were analyzed in addition to success rates. It was previously observed (Sec. 5.4.2) that running the solver with few iterations led to incorrect results; therefore, the tests thus far have been performed with a high number of iterations for both the non-linear (NLP) and quadratic (QP) problems. However, maintaining such high iteration counts significantly slows down the resolution, exceeding the defined time step of 20 ms by an order of magnitude (Tab. 5.9).

Horizon	**Average Time** [s]	**99-Percentile** [s]	**Max Time** [s]
$N = 10$	0.05651	0.14705	0.17341
$N = 15$	0.13191	0.17221	0.19115
$N = 20$	0.15614	0.19590	0.24720
$N = 25$	0.16488	0.21432	0.25432
$N = 30$	0.14489	0.22985	0.27546

Table 5.9: Computational times with both iteration high, without box creation and with $LD = 3.0$ m

Since the algorithm is intended to control a drone in real-time, computation times can be reduced by lowering the maximum iterations of the non-linear problems while keeping

the iterations for the quadratic case high[6]. With this approach, computational times drop drastically. In addition to the times specifically required by the solver (Tab. 5.10), the times including the safe-box creation by the LiDAR were also evaluated (Tab. 5.11) to provide a comprehensive estimate of a single step's duration. Regarding the LiDAR, to avoid increasing execution times, the sensor detects front and rear obstacles simultaneously, halving the required number of computed rays while maintaining the exact same physical resolution (in 2D, 180 rays and $\Delta\theta = 1°$).

Without Box Creation			
Horizon	**Average Time [s]**	**99-Percentile [s]**	**Max Time [s]**
$N = 10$	0.00749	0.00915	0.04374
$N = 15$	0.00903	0.01096	0.04402
$N = 20$	0.01008	0.01209	0.04577
$N = 25$	0.01108	0.01332	0.04291
$N = 30$	0.01188	0.01439	0.07822

Table 5.10: Computational times with high qp-iteration, without box creation and with $LD = 3.0$ m

With Box Creation			
Horizon	**Average Time [s]**	**99-Percentile [s]**	**Max Time [s]**
$N = 10$	0.01346	0.01581	0.03343
$N = 15$	0.01516	0.01771	0.08835
$N = 20$	0.01620	0.01861	0.02610
$N = 25$	0.01759	0.02149	0.08887
$N = 30$	0.01809	0.02669	0.21881

Table 5.11: Computational times with high qp-iteration, with box creation and with $LD = 3.0$ m

Observing from the tests that excessively short horizons cause the drone to take oscillatory and non-smooth trajectories, and needing to select the highest possible horizon with execution times remaining safely under the time step $dt = 20$ ms, the choice fell on the horizon $N = 20$.

5.5.2 Comparison with other state-of-the-art methods

To validate RBOC-MPC, we compare it against other state-of-the-art approaches for obstacle avoidance: Std-MPC, which corresponds to the MPC without terminal constraints, Zero-TC MPC, which substitutes the NN constraint with a zero terminal constraint for velocity and angles and the Artificial Potential Field (APF) method [6], which computes the drone's motion by immersing it in a force field and solving the resulting kinematics. In this comparison, the test environment features several randomly placed obstacles that are easy to overcome (e.g., short walls and moderate inclinations) and there is no specific position target: the drone moves linearly to the right, dodging obstacles along its path. The only target the drone has is a velocity target for the x-coordinate in 2D:

$$v_{target} = [v_x, 0] \qquad \text{(Eq. 5)}$$

[6]Max iterations: $nlp = 5$, $qp = 100$.

Cruising speeds close to those learned by the network were chosen so as not to cause large accelerations and decelerations in the drone's path: $v_{target} \in [0.8, 1.2]$ m/s. The goal is no longer to make the drone reach a specific point or to observe how it crosses obstacles, aspects already evaluated in previous tests. Instead, the aim is to evaluate the cost of the RBOC-MPC in relation to the percentage of successes and failures, and in contrast with other classical methods. The LiDAR range was set to an average value of 3 m for all tests, as was the horizon $N = 20$. The simulation times are identical across the various cases, as are the integration steps, in order to have the same total number of iterations.

Cost calculation

The weights in the cost function have been modified compared to the previous ones (Eq. 7) to give greater weight to the velocity target and zero out the position target:

$$W_x = \begin{bmatrix} 10^{-4} & 0 & 0 & 0 & 0 & 0 \\ 0 & 10^{-4} & 0 & 0 & 0 & 0 \\ 0 & 0 & 20 & 0 & 0 & 0 \\ 0 & 0 & 0 & 10^2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \qquad W_u = \begin{bmatrix} 10^{-4} & 0 \\ 0 & 10^{-4} \end{bmatrix} \quad \text{in 2D} \qquad \text{(Eq. 6)}$$

The cost function in the MPC includes both a running cost and a terminal cost (Eq. 6). However, the terminal cost is used solely to improve the solver's stability and does not actually contribute to the costs paid by the drone along the flown trajectory. For this reason, the only cost calculated in the three tested methods is the running one:

$$l(x_i, u_i) = \epsilon_{x_i}^T W_x \epsilon_{x_i} + \epsilon_{u_i}^T W_u \epsilon_{u_i} \qquad \text{(Eq. 7)}$$

It should be noted that in the APF case, the motor torque values are not known a priori, since the model to which the resulting force (Eq. 11) is applied is a point mass: $F_{res} = [F_{res,x}, F_{res,z}]$ in 2D. Discarding an analytical approach, it is possible to derive the accelerations along the two axes and, through kinematics, the future positions and velocities of the point mass:

$$\begin{aligned} a_x &= \frac{F_{res,x}}{m} & x_{target} &= x_{curr} + v_{x,curr}dt + \frac{1}{2}a_x dt^2 \\ a_z &= \frac{F_{res,z}}{m} & z_{target} &= z_{curr} + v_{z,curr}dt + \frac{1}{2}a_z dt^2 \\ & & v_{x,target} &= v_{x,curr} + a_x dt \\ & & v_{z,target} &= v_{z,curr} + a_z dt \end{aligned} \qquad \text{(Eq. 8)}$$

The latter become the target for a very short-horizon MPC optimizer (e.g., $N = 5$) containing the drone's dynamic model and only the constraints on motors, velocities, and angles. The MPC returns the motor torques, which are used in the cost calculation (Eq. 6), and the real position of the drone at the next step to continue the simulation with realistic values. This cascading procedure, which became necessary to adapt the simplified point-mass model to the real one, certainly reveals a first limitation of the APF method.

APF with velocity target

The classical APF method involves a position target (Eq. 10) which, as explained previously, is replaced by a velocity target. Assuming the drone must continue moving only

along the x-axis with a target velocity, even after reaching cruising speed, the attractive force is defined as follows:

$$F_{att} = \begin{bmatrix} F_{att,x} \\ F_{att,z} \end{bmatrix} = \begin{bmatrix} k_{att}(v_{target} - v_x) \\ k_{damp}(0 - v_z) \end{bmatrix} \quad \text{in 2D} \qquad \text{(Eq. 9)}$$

In this way, the algorithm uses a purely proportional control on the velocity difference along the X-axis: the drone accelerates when $v_x < v_{target,x}$ and brakes autonomously when $v_x > v_{target,x}$. When the velocity error reaches zero ($v_x = v_{target}$), the applied force becomes zero and the drone proceeds by inertia, maintaining the target. On the Z-axis, since a vertical velocity target is not provided, a damping term proportional to the gain k_{damp} has been inserted. This acts as a vertical brake to dissipate the kinetic energy accumulated while climbing over an obstacle, quickly returning the drone to stabilized horizontal flight ($v_z = 0$). Without this brake, it was observed that the drone does not proceed by maintaining a straight trajectory on the x-axis, but instead tends to bounce among the obstacles, retaining the kinetic energy accumulated in the z-direction.

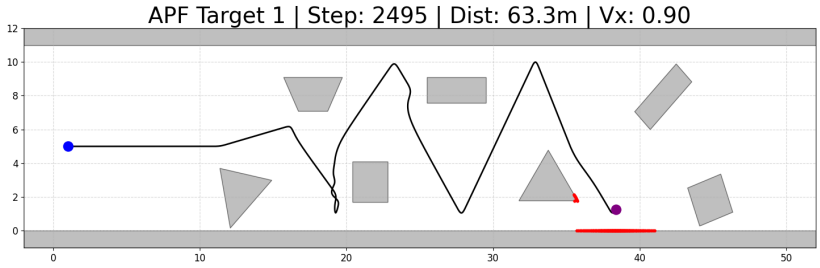


Figure 5.32: Example of an APF trajectory without kinetic energy dissipation in z

Obstacles are identified using the LiDAR function developed for the RBOC-MPC, maintaining the logic of local path planning. However, in this case, defining the boundaries of the safe box is not important. It is possible, though, to extract the intersections between the LiDAR and the obstacles, modeled as circles (Eq. 16) to fully resume the original method developed by Khatib [6]. A repulsive force is then defined from each circle in the direction given by the LiDAR itself (Eq. 10).

The parameters used for the simulation are the following:

Parameter	Value
k_{att}	2.5
k_{damp}	5.0
k_{rep}	0.05
ρ_0	3.0

Table 5.12: Parameters for APF

The parameters utilized for this algorithm (Tab. 5.12) were heuristically tuned to provide the best possible performance in the baseline scenario (Environment A).

Comparison results

The results obtained for 100 randomized tests with the four evaluated methods are reported below: RBOC-MPC, Std-MPC, Zero-TC MPC and APF. The target velocity (Eq. 5) has

an average value of 0.97 m/s, and the simulated environments feature two different obstacle densities, with obstacles shifting stochastically from one test to the next:

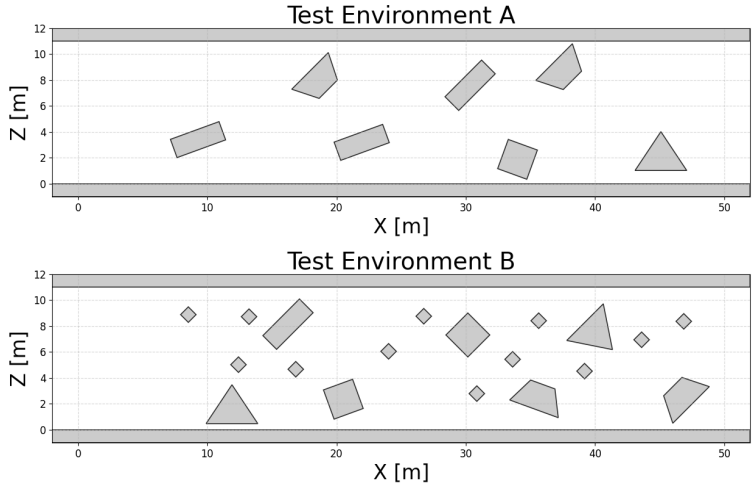


Figure 5.33: Examples of simulation environments A and B with varying obstacle densities

The results in Tab. 5.13 and Tab. 5.14 correspond to the average values of the cost, total distance traveled, and distance traveled along the x-axis, exclusively for the successful cases (exhaustion of all available iterations). In the event of a crash, the results are considered null and are excluded from the average calculation. Therefore, the reported average values must also be evaluated in relation to the success rate associated with the respective method.

Environment A				
Method	**Success %**	**Cost**	**Distance [m]**	**x-Distance [m]**
RBOC-MPC	100	11818.39	46.93	46.42
APF	100	53975.61	38.74	37.23
Std-MPC	18	5850.75	46.77	46.72
Zero-TC MPC	100	238849.0	1.85	1.83

Table 5.13: Mean values for three methods compared for 100 tests in environment A

Environment B				
Method	**Success %**	**Cost**	**Distance [m]**	**x-Distance [m]**
RBOC-NN	100	23345.88	45.79	44.61
APF	100	131495.77	27.83	25.54
Std-MPC	2	17144.89	47.71	47.33
Zero-TC MPC	100	238884.97	1.82	1.80

Table 5.14: Mean values for three methods compared for 100 tests in environment B

The tests confirm the validity of the RBOC-MPC method, which achieves a significantly higher success rate than the Std-MPC and incurs lower costs compared to the APF algorithm and Zero-TC MPC, which likely struggle to maintain the assigned target. Furthermore, RBOC-MPC covers on average greater distances than APF and Zero-TC MPC. APF suffers from larger deviations along the z-axis, instead Zero-TC MPC makes

the drone proceed very slowly, covering only a few meters. Although these differences are already noticeable in the first environment, they become even more prominent as the obstacle density increases. Furthermore, the APF method presents the modeling limitations previously introduced in the event that the point-like model needs to be adapted to more complex dynamics. A major limitation of the APF method is that its user-defined gains in Tab. 5.12 are highly environment-dependent: fixed parameters that perform well in sparse areas often lead to crashes or oscillatory behaviors when obstacle density increases (Environment B), as the superposition of repulsive forces changes drastically.

As can be seen from the results, in the case of Zero-TC MPC, maintaining the horizon at $N = 20$, ensures that the drone moves at very low speeds, traveling only a few meters and prioritizing a conservative braking approach over target velocity tracking. To overcome this issue, the horizon can be extended to $N = 30$ or $N = 40$; in this case, the zero terminal constraint effectively improves success rates compared to the unconstrained Std-MPC case. However, a comparison between methods with different time horizons would not be fair and would result in a significant increase in computational time. This intrinsic limitation of Zero-TC MPC further justifies the adoption of a safe set learned via a Neural Network, capable of guaranteeing safety at the same horizon ($N = 20$) without paralyzing the system's dynamics.

5.5.3 Dynamic Environments

To conclude this analysis, the validation was extended by modifying Environment A (Fig. 5.33) to include moving obstacles with intentionally limited velocities. Since the LiDAR function recalculates the safe-box at each iteration, the mapping approach is inherently suited for handling dynamic environments, updating the terminal constraint within the RBOC-MPC in real time. Over 100 randomized tests featuring moving obstacles and a velocity target, a success rate of 86% was recorded. The remaining 14% of cases resulted in failures caused by collisions with the obstacles. An analysis of these impacts revealed that they predominantly occur under unfavorable kinematic conditions, namely when the drone is effectively trapped, surrounded by moving obstacles with no physically feasible escape routes. Overall, these simulations confirm the validity and responsiveness of the mapping method even in the presence of dynamic obstacles, highlighting its potential adaptability to real-world operational scenarios.

Chapter 6

3D Adaptment

All the analyses and results deemed significant for the simplified 2D model were replicated for the 3D quadrotor case.

6.1 Redefinition of the learning dataset, NN training and validation

First, by redefining the quadcopter dynamics, the generation of the learning dataset and the training of the Neural Network were re-executed in that order for the case of a cubic box.

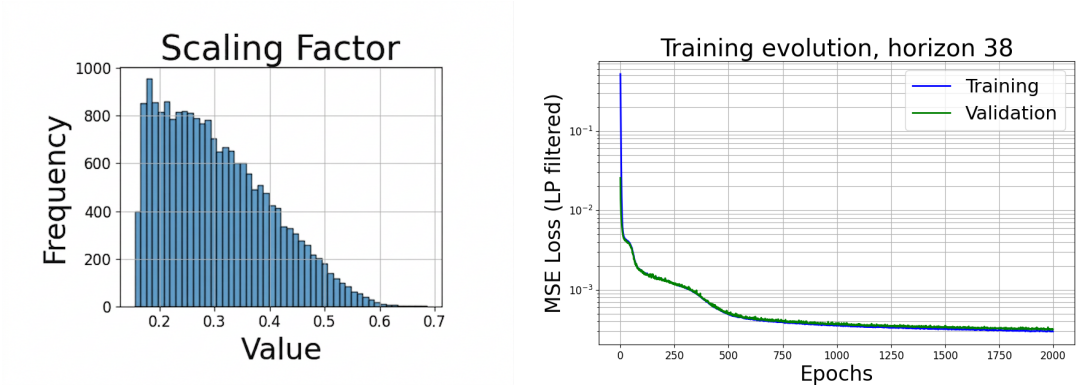


Figure 6.1: Distribution of Scaling Factor α and NN training evolution in 3D

In general, both in defining the initial set and during the various tests, it was observed that the 3D case requires longer prediction horizons N compared to the 2D case (e.g. $N = 38$ for the definition of the Viability Kernel).

The same analysis performed in Sec. 5.2.1 to evaluate the reliability of RBOC compared to other state-of-the-art methods, such as DeepReach, was also conducted in 3D. The comparison criteria and the metric used are exactly the same. Fig. 6.2 therefore shows the trade-off between false alarms and missed alarms for the quadcopter case. In this case, the DeepReach network was trained twice, and the results of both training runs are reported; it should be noted that, in both instances, the time required was significantly greater than for the RBOC case, as shown in Tab. 6.1.

Method	Space	Time
RBOC	3D	15 min
DeepReach	3D	10 h, 14 min (first attempt) 57 h, 30 min (second attempt)

Table 6.1: Computation times of RBOC and DeepReach

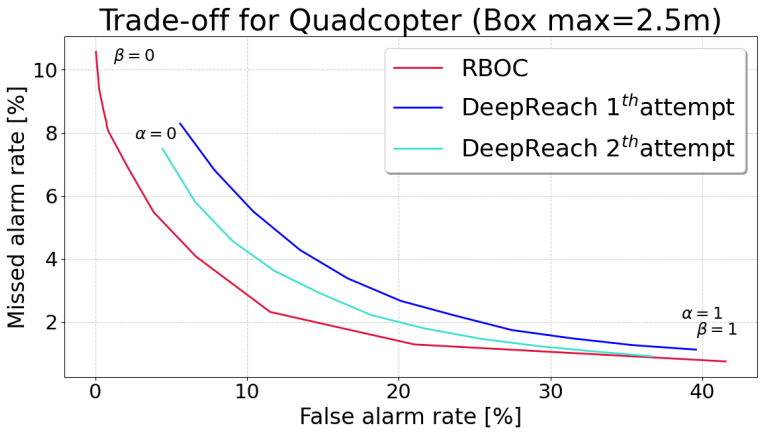


Figure 6.2: Performance trade-off between False Alarm Rate and Missed Alarm Rate for the Quadcopter model

The Quadcopter system, being dynamically more complex, presents higher error rates for both methods. RBOC in its nominal configuration ($\beta = 0$) is less conservative than DeepReach (with $\alpha = 0$), recording a missed-alarm rate (potential crashes) $> 10\%$. However, as soon as β is increased, RBOC exhibits an extremely rapid error decay. Even with a minimal tolerance (e.g., $\beta \approx 0.18$ and False alarm rate $\approx 6\%$), RBOC outperforms DeepReach. For higher values of β, the RBOC curve consistently stays below both the first and second DeepReach attempts, confirming the good performance already observed with the Bicopter (Tab. 5.3).

6.2 RBOC-MPC application in 3D environments

6.2.1 Preliminary tests on axis-aligned obstacles

We then proceeded to define the RBOC-MPC for the testing and validation phase of the learning process. In this case as well, the initial mapping was performed on axis-aligned obstacles, limiting the LiDAR range and handling both stalls and infeasibility (Status 4) using the exact same approach as in the 2D case[1]. Fig. 6.3 shows the path taken by the drone in a labyrinthine scenario, similar to the one tested in the two-dimensional environment (see Fig. 5.19).

[1]In 3D, $W = 20$ for axis-aligned obstacle

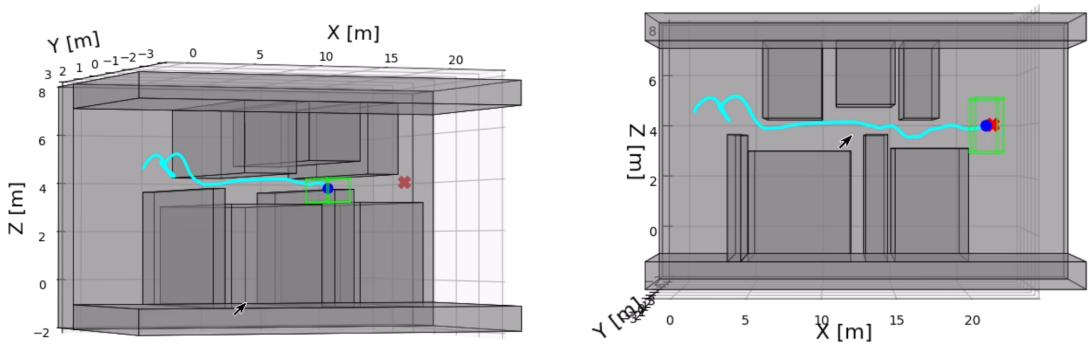


Figure 6.3: Drone performance in an environment with axis-aligned obstacles

6.2.2 Generalization of polygonal obstacle mapping in 3D

In adapting the mapping to general obstacles, the core idea is to identify the obstacles by intersecting the LiDAR ray with the obstacle faces, which are geometrically defined by planar surfaces. In general, 4-vertex polygons may not be coplanar; this is why a triangulation method known as the Möller–Trumbore algorithm is adopted [17].

A triangle in 3D space is defined by three vertices A, B, and $C \in \mathbb{R}^3$. From these, two edge vectors are derived:

$$\begin{aligned} E_1 &= B - A \\ E_2 &= C - A \end{aligned} \qquad \text{(Eq. 1)}$$

Any point within the triangle is described by the barycentric coordinates u and v, such that:

$$S(u, v) = A + uE_1 + vE_2 \qquad \text{(Eq. 2)}$$

The equation of the LiDAR line remains the same as previously used in the 2D case (Eq. 11), and the impact occurs when the two geometric figures (the line Eq. 11 and the triangle Eq. 2) intersect, resulting in a linear system of 3 equations with 3 unknowns (t, u, v):

$$\begin{cases} -td_x + uE_{1,x} + vE_{2,x} = P_{0,x} - A_x = \Delta_x \\ -td_y + uE_{1,y} + vE_{2,y} = P_{0,y} - A_y = \Delta_y \\ -td_z + uE_{1,z} + vE_{2,z} = P_{0,z} - A_z = \Delta_z \end{cases} \quad \text{in 3D} \qquad \text{(Eq. 3)}$$

Solving the system, exactly as in the 2D case, the intersection is physically valid only if it respects the following mathematical constraints: $t \geq 0$, $u \geq 0$, $v \geq 0$, and $u + v \leq 1$.

In this case as well, Fig. 6.4 shows an example of a trajectory followed by the drone in an irregular environment.

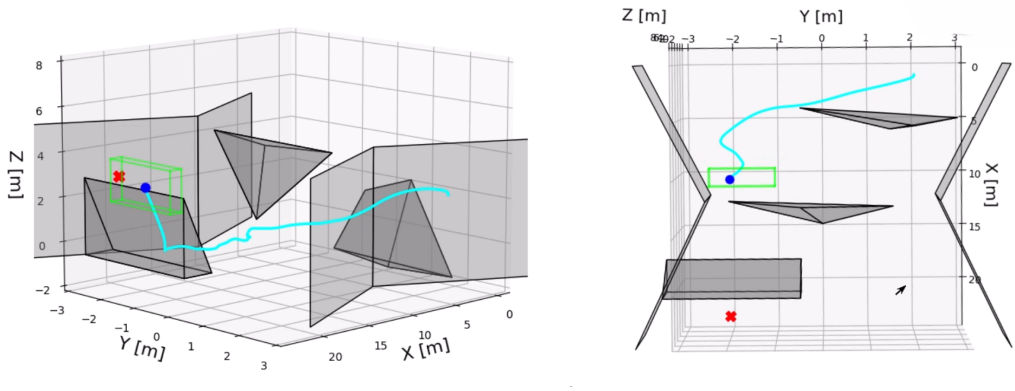


Figure 6.4: Drone performance in an environment with general obstacles

Chapter 7

Conclusions

This thesis addressed the challenge of safe UAV navigation in unknown environments, a particularly complex problem for underactuated systems such as quadrotors. Traditional approaches based on Model Predictive Control (MPC) offer excellent performance in trajectory tracking, but suffer from short-sightedness over short predictive horizons, leading to recursive feasibility issues and potential collisions. To overcome this limitation, this work extended the Viability Boundary Optimal Control (VBOC) method to drones, defining the Recoverability Boundary Optimal Control (RBOC) framework and introducing the concept of a "recovery box": a safe space within which the drone, starting from randomized initial conditions, is able to come to a safe stop. A dataset was generated via offline Optimal Control Problems (OCPs), recording the dimensions of the recovery boxes and the initial conditions of the drone, which was subsequently used to train a Neural Network via supervised learning. In this way, the system is able to estimate the minimum braking space required by the vehicle based solely on its kinematics. Finally, the network was integrated into the MPC as a differentiable terminal constraint, guaranteeing that the predicted stopping space is always strictly contained within an obstacle-free safe-box, extracted in real-time using a mathematically simulated LiDAR sensor.

The simulation results confirm the effectiveness and superiority of the proposed framework compared to other state-of-the-art methods. In the purely predictive safety validation, RBOC demonstrated a drastically lower missed-alarm rate compared to complex techniques based on Hamilton-Jacobi reachability (DeepReach), while requiring offline computational times several orders of magnitude smaller. Numerous intermediate tests with position targets led to the definitive method for safe-box extraction, leveraging a Warm-Start approach; concurrently, the LiDAR mapping, initially implemented for axis-aligned obstacles, was successfully extended to generic polygonal shapes. Furthermore, in navigation tests with a velocity target, the RBOC-MPC eliminated physical collisions compared to the Standard-MPC, robustly managing infeasibility issues. The method proved to perform significantly better than classical reactive algorithms such as Artificial Potential Fields (APF) or MPC with zero terminal constraints (Zero-TC MPC), ensuring a lower operational cost and a faster, more continuous progress toward the target. A further significant achievement of this work is the scalability of the system: initially validated on a 2D model (bicopter), the framework was successfully extended to the full 3D dynamics of a quadrotor.

Despite the promising results, the work presents limitations that pave the way for future developments. First, theoretical guarantees of recursive feasibility do not automatically transfer in the presence of an imperfect approximation by the Neural Network. Future work should therefore investigate certified inner approximations of the recoverabil-

ity set and the introduction of robust margins to compensate for modeling uncertainties and sensor inaccuracies. Second, simulations highlighted that a purely local planning approach centered on position targets can lead to path inefficiencies, pushing the drone to undertake suboptimal trajectories, causing temporary stalls in front of large obstacles, or generating solver convergence issues in the presence of sudden sharp corners. To overcome these issues, several compensatory techniques were introduced, such as autonomous stall management, the activation of recovery procedures, and the introduction of a safety margin in the safe-box creation. Although these solutions were implemented ad hoc to mitigate the criticalities observed in simulation, they cannot be considered definitive. It should be emphasized, however, that the focus of this thesis is safety rather than path planning; many of the observed limitations stem inherently from the purely local nature of LiDAR mapping. The integration of a Global Planner or the inclusion of suitably calculated waypoints would resolve most of these inefficiencies without compromising the developed safety architecture in the slightest.

Additional research directions include extending the representation of the local free space to more complex geometries, such as the union of multiple boxes or general convex polytopes, thus overcoming the constraint of a strictly rectangular or cuboid shape. Finally, the most significant step suggested by this work is the experimental validation on real hardware architectures of what has been developed in simulation. In this regard, it must be noted that the dynamic model employed here is ideal, as it neglects aerodynamic drag and assumes an instantaneous response from the actuators. For application on real physical systems, such dynamics (e.g., motor time constants) will need to be modeled, and the control parameters recalibrated to adapt to actual flight conditions.

Bibliography

[1] J. Li, X. Xiong, Y. Yan, and Y. Yang, "A survey of indoor uav obstacle avoidance research," *IEEE Access*, vol. 11, pp. 51861–51891, 2023.

[2] G. Lunardi, A. L. Rocca, M. Saveriano, and A. D. Prete, "Receding-constraint model predictive control using a learned approximate control-invariant set," *arXiv preprint / conference submission*, 2024. University of Trento.

[3] E. Fontanari, G. Lunardi, M. Saveriano, and A. D. Prete, "Parallel-constraint model predictive control: Exploiting parallel computation for improving safety," *arXiv preprint / conference submission*, 2025. University of Trento.

[4] A. D. Prete, "Invited talk: Safe robot control combining learning and model-based control." `https://www.youtube.com/watch?v=_oMz9_LaFCc`, 2023. YouTube video, accessed: 2026-04-19.

[5] J. Sun, J. Tang, and S. Lao, "Collision avoidance for cooperative uavs with optimized artificial potential field algorithm," *IEEE Access*, vol. 5, pp. 18382–18390, 2017.

[6] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.

[7] L. G. Carrillo, A. D. López, R. Lozano, and C. Pégard, *Modeling the Quad-Rotor Mini-Rotorcraft*, ch. 2, pp. 23–34. London: Springer-Verlag London, 2013.

[8] S. Bansal and C. J. Tomlin, "Deepreach: A deep learning approach to high-dimensional reachability," in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1817–1824, IEEE, 2021.

[9] J. Li, D. Lee, J. Lee, K. S. Dong, S. Sojoudi, and C. Tomlin, "Certifiable reachability learning using a new lipschitz continuous value function," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3582–3589, 2025.

[10] M. S. Asia La Rocca and A. D. Prete, "Vboc: Learning the viability boundary of a robot manipulator using optimal control," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 6971–6978, 2023.

[11] T. Akenine-Möller, E. Haines, and N. Hoffman, *Real-Time Rendering*. Boca Raton, FL: CRC Press, 4 ed., 2018.

[12] A. Williams, S. Barrus, R. K. Morley, and P. Shirley, "An efficient and robust ray–box intersection algorithm," *Journal of Graphics Tools*, vol. 10, no. 1, pp. 49–54, 2005.

[13] M. Bertoni, "safe-mpc." `https://github.com/MaxBertus/safe-mpc`, 2026. GitHub repository.

[14] M. Bertoni, "VBOC." `https://github.com/MaxBertus/VBOC`, 2026. GitHub repository.

[15] C. Ericson, *Real-Time Collision Detection*. The Morgan Kaufmann Series in Interactive 3D Technology, Morgan Kaufmann, 2005.

[16] E. Fontanari, "Deepreach (branch: Deepreach mpc)." `https://github.com/EliasFontanari/deepreach`, 2026.

[17] T. Möller and B. Trumbore, "Fast, minimum storage ray–triangle intersection," *Journal of Graphics Tools*, vol. 2, no. 1, pp. 21–28, 1997.